Учебник по 27 признакам EH/Unwind Analyzer

Второе издание: связное объяснение от нуля до уверенного чтения static findings, отчетов, Ghidra и dynamic evidence.

Этот учебник написан не как шпаргалка, а как путь обучения. Он начинается с простых понятий - файл, байты, адрес, секция - и постепенно приводит к пониманию всех 27 признаков. После чтения ты должен уметь открыть HTML или JSON отчет, понять почему finding появился, объяснить affected count и показать место в Ghidra.

Главная мысль всей книги: EH/Unwind Analyzer не угадывает вредоносность. Он читает конкретную metadata Windows x64 и превращает наблюдаемые нарушения в объяснимые findings. Это делает вывод проверяемым: можно открыть адрес, увидеть секцию, сравнить bytes и понять, насколько сильный case.

Раздел	Что должен понять читатель
Часть I. Нулевая база	что такое PE, секции, адреса, RUNTIME_FUNCTION и UNWIND_INFO
Часть II. Как думать о findings	разница между finding count, affected count, score, confidence и evidence
Часть III. 27 признаков	глубокое объяснение каждого public finding проекта
Часть IV. Практика	как читать combo_28, static_03, JSON, HTML и Ghidra CSV

Часть I. Нулевая база: от файла к unwind metadata

Файл как последовательность байтов

Любой EXE на диске сначала является обычным файлом. Внутри него нет функций в человеческом смысле, нет красивых таблиц и нет подсказок для Ghidra. Есть только последовательность байтов. Чтобы превратить эти байты в структуру, программа должна знать формат. Для Windows EXE/DLL этим форматом является PE - Portable Executable.

Parser не имеет права доверять байтам просто потому, что файл называется .exe. Он проверяет заголовки, размеры, смещения и секции. Если поле говорит 'данные лежат по смещению 0x5000', parser должен убедиться, что файл действительно имеет такие байты. Иначе любое дальнейшее чтение превратится в фантазию.

Именно поэтому первые findings проекта связаны не с исключениями, а с PE layout. Если секции выходят за границы файла или пересекаются, analyzer сначала фиксирует это. Только после этого можно честно говорить о .pdata и UNWIND_INFO.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
байт	минимальная единица данных в файле. EXE на диске - это последовательность байтов, которую программа-парсер читает по смещениям.
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
VA	Virtual Address, полный виртуальный адрес в памяти процесса. Обычно VA = ImageBase + RVA.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.

Адреса: почему RVA и VA постоянно путают

RVA - это адрес относительно начала загруженного образа. VA - это адрес в памяти процесса. Разница кажется мелочью, пока ты не открываешь Ghidra или debugger. Analyzer обычно сообщает RVA, потому что RVA стабилен внутри файла. Debugger показывает VA, потому что процесс уже загружен в память.

Формула простая: VA = ImageBase + RVA. Если ImageBase равен 0x140000000, а RVA равен 0x6000, то адрес в Ghidra обычно будет 0x140006000. Ошибка в этой формуле приводит к тому, что человек смотрит не туда и делает неверный вывод.

В отчетах проекта representative_rva - это не полный runtime address. Это относительный адрес. Его удобно хранить в JSON и HTML, потому что он не зависит от ASLR. При ручной проверке его надо переводить в адрес инструмента, которым ты пользуешься.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
байт	минимальная единица данных в файле. EXE на диске - это последовательность байтов, которую программа-парсер читает по смещениям.
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
VA	Virtual Address, полный виртуальный адрес в памяти процесса. Обычно VA = ImageBase + RVA.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.

Секции и права R/W/X

Секция PE похожа на контейнер с ролью. .text обычно содержит исполняемый код. .pdata содержит RUNTIME_FUNCTION table. .xdata содержит UNWIND_INFO. .rdata содержит read-only data. Это не железный закон для всех компиляторов мира, но это нормальная ожидаемая раскладка.

Права секции объясняют, что loader разрешает делать с памятью: R - читать, W - писать, X - исполнять. Для metadata нормальная интуиция такая: данные надо читать, но обычно не надо

исполнять и не надо менять после загрузки. Поэтому writable или executable placement для unwind metadata требует проверки.

Когда analyzer пишет unwind section unusual или unwind section writable, он не говорит, что структура битая. Он говорит: структура может быть корректной, но лежит в месте, которое плохо похоже на обычную read-only metadata.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
байт	минимальная единица данных в файле. EXE на диске - это последовательность байтов, которую программа-парсер читает по смещениям.
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
VA	Virtual Address, полный виртуальный адрес в памяти процесса. Обычно VA = ImageBase + RVA.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.

Что такое RUNTIME_FUNCTION

В Windows x64 каждая не-trivial функция должна быть описана так, чтобы система могла раскрутить стек при исключении. Для этого используется таблица RUNTIME_FUNCTION. Каждая запись хранит начало функции, конец функции и адрес UNWIND_INFO.

Эта таблица обычно находится в Exception Directory, а физически часто связана с секцией .pdata. Если запись говорит, что функция начинается в 0x1010 и заканчивается в 0x1149, analyzer может показать этот диапазон в Runtime functions table и передать его в Ghidra CSV.

Если begin больше end, если диапазоны пересекаются, если таблица не отсортирована или указывает за пределы образа, это уже не косметика. Это ломает карту функций, на которую опираются debugger, decompiler и ручной анализ.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
байт	минимальная единица данных в файле. EXE на диске - это последовательность байтов, которую программа-парсер читает по смещениям.
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
VA	Virtual Address, полный виртуальный адрес в памяти процесса. Обычно VA = ImageBase + RVA.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.

Что такое UNWIND_INFO

UNWIND_INFO - это инструкция для Windows: что функция сделала со стеком и регистрами в прологе, чтобы при исключении можно было восстановить состояние. В ней есть version, flags, prolog size, count of unwind codes, optional handler и sometimes chained info.

Если UNWIND_INFO корректна, analyzer может вывести stack size, saved registers, flags и handler_rva. Если она оборвана или указывает на невозможный хвост, появляется parse failure. Если она читается, но содержит странные коды или ABI deviations, появляются unwind anomalies или strict ABI diagnostics.

Понимание UNWIND_INFO важно для reverse engineering. Декомпилятор может ошибиться в границах функции или stack frame, если metadata говорит неправду. Поэтому проект фокусируется именно на этом низкоуровневом слое.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение

байт	минимальная единица данных в файле. EXE на диске - это последовательность байтов, которую программа-парсер читает по смещениям.
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
VA	Virtual Address, полный виртуальный адрес в памяти процесса. Обычно VA = ImageBase + RVA.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.

Часть II. Как думать о findings

Finding count и affected count

Finding count отвечает на вопрос: сколько разных правил сработало. Affected count отвечает на другой вопрос: сколько объектов попало под конкретное правило. Если один finding затронул 46 функций, это не значит, что у нас 46 разных аномалий. Это значит, что один pattern распространился на 46 entries.

Пример combo_28 как раз такой. Сработали два findings: unwind_unusual_section и unwind_writable_section. Каждый имеет affected count 46, потому что все 46 RUNTIME_FUNCTION entries указывают на UNWIND_INFO в секции .weird с правом записи.

Эта разница спасает от неправильной презентации. Правильно говорить: 'сработало два типа признаков, каждый массово затронул 46 функций'. Неправильно говорить: 'нашлось 92 разных ошибки'.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
score	балл подозрительности 0..100. Это не вероятность, а сумма весов findings.
confidence	уверенность в интерпретации. Она зависит от количества, силы и разнообразия evidence.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.
static analysis	анализ без запуска файла. Analyzer читает PE и metadata как данные.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.

Score, suspicion и confidence

Score - это инженерный балл подозрительности. Он складывается из весов findings и ограничивается диапазоном 0..100. Он не является вероятностью. Нельзя говорить 'score 33 значит 33 процента malware'. Это неверно.

Suspicion - нормализованная форма score, то есть score / 100. Она удобна для JSON и интерфейсов, но смысл остается тем же: это шкала ранжирования, а не статистическая вероятность.

Confidence - отдельная идея. Она отвечает, насколько уверенно можно интерпретировать case. Один слабый finding дает маленькую confidence. Несколько high findings из разных families дают большую confidence. Dynamic correlation тоже может усилить итог.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
score	балл подозрительности 0..100. Это не вероятность, а сумма весов findings.
confidence	уверенность в интерпретации. Она зависит от количества, силы и разнообразия evidence.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.
static analysis	анализ без запуска файла. Analyzer читает PE и metadata как данные.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.

Почему HTML не показывает все 27 строк всегда

HTML-отчет создан для чтения человеком. Если в каждом отчете показывать все 27 findings, включая нулевые, важные строки утонут в шуме. Поэтому HTML показывает сработавшие findings и summary.

JSON создан для полной машины картины. Там есть counters, и многие из них равны нулю. Если надо доказать, что проверка выполнялась, открывай heuristic_analysis в JSON.

Документация docs/evidence_matrix.md и этот учебник отвечают за полный список правил. Конкретный HTML отвечает за конкретный sample.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.

score	балл подозрительности 0..100. Это не вероятность, а сумма весов findings.
confidence	уверенность в интерпретации. Она зависит от количества, силы и разнообразия evidence.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.
static analysis	анализ без запуска файла. Analyzer читает PE и metadata как данные.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.

Static и dynamic evidence

Static evidence - это то, что видно в файле без запуска. Dynamic evidence - это то, что target реально сделал при запуске под Debug API. Эти слои связаны, но не заменяют друг друга.

Если static finding говорит, что metadata writable, dynamic mode может показать вызов RtlAddFunctionTable или handler hit. Тогда case становится сильнее. Но отсутствие dynamic hit не уничтожает static finding: возможно, путь выполнения просто не был достигнут.

Проект специально разделяет static_evidence_level и runtime_evidence_level, а затем делает final assessment. Это честнее, чем смешивать все в один мутный verdict.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
score	балл подозрительности 0..100. Это не вероятность, а сумма весов findings.
confidence	уверенность в интерпретации. Она зависит от количества, силы и разнообразия evidence.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.
static analysis	анализ без запуска файла. Analyzer читает PE и metadata как данные.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.

Часть III. Подробный разбор всех 27 признаков

Дальше каждая глава устроена одинаково: сначала смысл признака, затем место в общей цепочке, затем почему это подозрительно, где это видно в отчете, как проверять руками и как объяснять на защите. Такая повторяемая форма нужна не для набивки страниц, а для обучения: мозг начинает видеть один и тот же аналитический шаблон на разных типах нарушений.

Глава 1. Признак pe_section_raw_oob

Эта глава разбирает finding pe_section_raw_oob. Его короткий смысл: секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_section_raw_oob: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_section_raw_oob становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +25. Confidence effect: strong malformed-file signal. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_section_raw_oob. Он означает, что секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_section_raw_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_section_raw_oob: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 2. Признак pe_section_virtual_oob

Эта глава разбирает finding pe_section_virtual_oob. Его короткий смысл: виртуальный диапазон секции выходит за объявленный SizeOfImage. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_section_virtual_oob: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_section_virtual_oob становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

> *Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.*

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +20. Confidence effect: strong image-layout signal. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_section_virtual_oob. Он означает, что виртуальный диапазон секции выходит за объявленный SizeOfImage. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage',

затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_section_virtual_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'виртуальный диапазон секции выходит за объявленный SizeOfImage', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_section_virtual_oob: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 3. Признак pe_section_raw_overlap

Эта глава разбирает finding pe_section_raw_overlap. Его короткий смысл: две секции пересекаются в файле и могут трактовать одни байты по-разному. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_section_raw_overlap: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +15. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_section_raw_overlap становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +15. Confidence effect: contributes through family and medium count. Mitigations: small_sample_size` only when otherwise isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_section_raw_overlap. Он означает, что две секции пересекаются в файле и могут трактовать одни байты по-разному. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты

по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_section_raw_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в файле и могут трактовать одни байты по-разному', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_section_raw_overlap: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 4. Признак pe_section_virtual_overlap

Эта глава разбирает finding pe_section_virtual_overlap. Его короткий смысл: две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_section_virtual_overlap: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_section_virtual_overlap становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +20. Confidence effect: strong ambiguous-mapping signal. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_section_virtual_overlap. Он означает, что две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_section_virtual_overlap это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_section_virtual_overlap: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 5. Признак pe_section_misaligned

Эта глава разбирает finding pe_section_misaligned. Его короткий смысл: адреса или размеры секций не согласованы с file/section alignment. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_section_misaligned: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +8. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_section_misaligned становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +8. Confidence effect: weak-to-medium structural signal. Mitigations: single-family mitigations may apply. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_section_misaligned. Он означает, что адреса или размеры секций не согласованы с file/section alignment. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем

объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_section_misaligned это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'адреса или размеры секций не согласованы с file/section alignment', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_section_misaligned: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 6. Признак pe_exception_directory_oob

Эта глава разбирает finding pe_exception_directory_oob. Его короткий смысл: Exception Directory находится вне допустимого образа или не переводится в валидные байты. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для pe_exception_directory_oob: PE layout. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно pe_exception_directory_oob становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +25. Confidence effect: strong exception-directory integrity signal. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding pe_exception_directory_oob. Он означает, что Exception Directory находится вне допустимого образа или не переводится в валидные байты. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для pe_exception_directory_oob это означает следующее: сначала надо назвать слой PE layout, затем показать факт 'Exception Directory находится вне допустимого образа или не переводится в валидные байты', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится pe_exception_directory_oob: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 7. Признак no_runtime_functions

Эта глава разбирает finding no_runtime_functions. Его короткий смысл: для AMD64 образа не найдено записей RUNTIME_FUNCTION. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для no_runtime_functions: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно no_runtime_functions становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: weak standalone signal because a leaf-only image may legitimately have no entries. Mitigations: single_signal_only`, `small_sample_size`, `single_family_only. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding no_runtime_functions. Он означает, что для AMD64 образа не найдено записей RUNTIME_FUNCTION. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для no_runtime_functions это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'для AMD64 образа не найдено записей RUNTIME_FUNCTION', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится no_runtime_functions: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 8. Признак invalid_ranges

Эта глава разбирает finding invalid_ranges. Его короткий смысл: в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для invalid_ranges: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=1%`, else `high, score weight = +10 / +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно invalid_ranges становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +20. Confidence effect: +0.08` via hard table-integrity bonus. Mitigations: small_sample_size` (only if isolated and non-strong). Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding invalid_ranges. Он означает, что в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для invalid_ranges это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится invalid_ranges: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 9. Признак rf_oob

Эта глава разбирает finding rf_oob. Его короткий смысл: диапазон функции выходит за границы валидного image/section context. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для rf_oob: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=1%`, else `high, score weight = +10 / +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно rf_oob становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +25. Confidence effect: +0.08` via hard table-integrity bonus. Mitigations: small_sample_size` (only if isolated and non-strong). Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding rf_oob. Он означает, что диапазон функции выходит за границы валидного image/section context. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context',

затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для rf_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазон функции выходит за границы валидного image/section context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится rf_oob: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 10. Признак unwind_rva_oob

Эта глава разбирает finding unwind_rva_oob. Его короткий смысл: адрес UNWIND_INFO не отображается в валидный file offset или секцию. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_rva_oob: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=1%`, else `high, score weight = +10 / +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_rva_oob становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +25. Confidence effect: +0.08` via hard table-integrity bonus. Mitigations: small_sample_size` (only if isolated and non-strong). Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_rva_oob. Он означает, что адрес UNWIND_INFO не отображается в валидный file offset или секцию. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_rva_oob это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'адрес UNWIND_INFO не отображается в валидный file offset или секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_rva_oob: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 11. Признак rf_unsorted

Эта глава разбирает finding rf_unsorted. Его короткий смысл: таблица функций не отсортирована по BeginAddress. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для rf_unsorted: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +15. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно rf_unsorted становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +15. Confidence effect: contributes through family count and medium count. Mitigations: small_sample_size`, `single_family_only` when isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding rf_unsorted. Он означает, что таблица функций не отсортирована по BeginAddress. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет

найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для rf_unsorted это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'таблица функций не отсортирована по BeginAddress', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится rf_unsorted: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 12. Признак rf_overlaps

Эта глава разбирает finding rf_overlaps. Его короткий смысл: диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для rf_overlaps: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно rf_overlaps становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +25. Confidence effect: +0.08` via hard table-integrity bonus. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding rf_overlaps. Он означает, что диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для rf_overlaps это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится rf_overlaps: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 13. Признак duplicate_begin

Эта глава разбирает finding duplicate_begin. Его короткий смысл: несколько записей имеют одинаковое начало функции. Он относится к семье table_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для duplicate_begin: RUNTIME_FUNCTION table. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно duplicate_begin становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: contributes through family count and medium count. Mitigations: small_sample_size`, `single_family_only` when isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding duplicate_begin. Он означает, что несколько записей имеют одинаковое начало функции. Finding относится к family table_integrity, то есть к слою: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет

найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для duplicate_begin это означает следующее: сначала надо назвать слой RUNTIME_FUNCTION table, затем показать факт 'несколько записей имеют одинаковое начало функции', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится duplicate_begin: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 14. Признак unwind_parse_fail

Эта глава разбирает finding unwind_parse_fail. Его короткий смысл: UNWIND_INFO невозможно разобрать до конца по формату Windows x64. Он относится к семье unwind_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_parse_fail: UNWIND_INFO parser. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=1%`, else `high, score weight = +10 / +25. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_parse_fail становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird,

то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +25. Confidence effect: +0.08` via hard unwind-integrity bonus. Mitigations: small_sample_size` (only if isolated and non-strong). Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_parse_fail. Он означает, что UNWIND_INFO невозможно разобрать до конца по формату Windows x64. Finding относится к family unwind_integrity, то есть к слою: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет

найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_parse_fail это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO невозможно разобрать до конца по формату Windows x64', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_parse_fail: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 15. Признак unwind_anomalies

Эта глава разбирает finding unwind_anomalies. Его короткий смысл: UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern. Он относится к семье unwind_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_anomalies: UNWIND_INFO parser. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=5%`, else `high, score weight = +10 / +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_anomalies становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird,

то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +20. Confidence effect: +0.08` via hard unwind-integrity bonus. Mitigations: small_sample_size` (only if isolated and non-strong). Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_anomalies. Он означает, что UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern. Finding относится к family unwind_integrity, то есть к слою: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain

и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_anomalies это означает следующее: сначала надо назвать слой UNWIND_INFO parser, затем показать факт 'UNWIND_INFO читается, но содержит странности в version,

opcode, operand, chain или ABI pattern', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_anomalies: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 16. Признак unwind_reuse_moderate

Эта глава разбирает finding unwind_reuse_moderate. Его короткий смысл: заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов. Он относится к семье unwind_reuse. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_reuse_moderate: reuse statistics. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_reuse_moderate становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: emitted only when another static signal corroborates moderate reuse. Mitigations: single_family_only` if remaining evidence is weak. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_reuse_moderate. Он означает, что заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов. Finding относится к family unwind_reuse, то есть к слою: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_reuse_moderate это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_reuse_moderate: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 17. Признак unwind_reuse_strong

Эта глава разбирает finding unwind_reuse_strong. Его короткий смысл: слишком большая доля функций использует один и тот же UnwindInfoRVA. Он относится к семье unwind_reuse. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_reuse_strong: reuse statistics. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_reuse_strong становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +20. Confidence effect: contributes through family count and strong count. Mitigations: usually not weakened. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_reuse_strong. Он означает, что слишком большая доля функций использует один и тот же UnwindInfoRVA. Finding относится к family unwind_reuse, то есть к слою: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет

найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_reuse_strong это означает следующее: сначала надо назвать слой reuse statistics, затем показать факт 'слишком большая доля функций использует один и тот же UnwindInfoRVA', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_reuse_strong: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 18. Признак handler_dominance

Эта глава разбирает finding handler_dominance. Его короткий смысл: один handler target обслуживает подозрительно много функций. Он относится к семье handler_dominance. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков смотрит на обработчики исключений. Если один handler обслуживает слишком много функций, аналитик должен понять, это нормальный runtime helper или централизованный exception dispatcher.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для handler_dominance: handler statistics. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +15. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно handler_dominance становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +15. Confidence effect: normal executable runtime handlers require corroborating static context. Mitigations: single_family_only` if remaining evidence is weak. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'B binary сработал finding handler_dominance. Он означает, что один handler target обслуживает подозрительно много функций. Finding относится к family handler_dominance, то есть к слою: Эта семья признаков смотрит на обработчики исключений. Если один handler обслуживает слишком много функций, аналитик должен понять, это нормальный runtime helper или централизованный exception dispatcher. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain

и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для handler_dominance это означает следующее: сначала надо назвать слой handler statistics, затем показать факт 'один handler target обслуживает подозрительно много функций', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится handler_dominance: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 19. Признак unwind_in_exec_section

Эта глава разбирает finding unwind_in_exec_section. Его короткий смысл: UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_in_exec_section: section placement. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +20. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_in_exec_section становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +20. Confidence effect: contributes through family count and strong count. Mitigations: no placement-only mitigation. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_in_exec_section. Он означает, что UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_in_exec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_in_exec_section: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 20. Признак unwind_unusual_section

Эта глава разбирает finding unwind_unusual_section. Его короткий смысл: UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_unusual_section: section placement. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=5%`, else `high, score weight = +8 / +15. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_unusual_section становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +8 / +15. Confidence effect: contributes through family count and medium/strong count. Mitigations: placement_only_signal`, `single_signal_only`, `small_sample_size`, `single_family_only. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_unusual_section. Он означает, что UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_unusual_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_unusual_section: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 21. Признак unwind_writable_section

Эта глава разбирает finding unwind_writable_section. Его короткий смысл: UNWIND_INFO лежит в секции с правом записи. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для unwind_writable_section: section placement. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium` if `<=5%`, else `high, score weight = +10 / +18. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно unwind_writable_section становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird,

то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10 / +18. Confidence effect: writable unwind metadata is stronger than merely unusual placement. Mitigations: single_signal_only`, `small_sample_size`, `single_family_only` when isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding unwind_writable_section. Он означает, что UNWIND_INFO лежит в секции с правом записи. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не

похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для unwind_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'UNWIND_INFO лежит в секции с правом записи', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится unwind_writable_section: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 22. Признак handler_nonexec_section

Эта глава разбирает finding handler_nonexec_section. Его короткий смысл: handler RVA указывает в неисполняемую или неизвестную секцию. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для handler_nonexec_section: section placement. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +15. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно handler_nonexec_section становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +15. Confidence effect: contributes through family count and strong count. Mitigations: usually not weakened. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding handler_nonexec_section. Он означает, что handler RVA указывает в неисполняемую или неизвестную секцию. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную

секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для handler_nonexec_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в неисполняемую или неизвестную секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится handler_nonexec_section: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 23. Признак handler_writable_section

Эта глава разбирает finding handler_writable_section. Его короткий смысл: handler RVA указывает в writable секцию. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для handler_writable_section: section placement. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно handler_writable_section становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: writable handler target is suspicious, especially when paired with other placement signals. Mitigations: single_family_only` when isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'B binary сработал finding handler_writable_section. Он означает, что handler RVA указывает в writable секцию. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain

и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется

правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для handler_writable_section это означает следующее: сначала надо назвать слой section placement, затем показать факт 'handler RVA указывает в writable секцию', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится handler_writable_section: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 24. Признак placement_exec_nonexec_handler_combo

Эта глава разбирает finding placement_exec_nonexec_handler_combo. Его короткий смысл: одновременно странно размещены unwind metadata и handler target. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для placement_exec_nonexec_handler_combo: combined evidence. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно placement_exec_nonexec_handler_combo становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

> *Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.*

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: raises confidence through extra strong evidence when both placement sides are suspicious. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding placement_exec_nonexec_handler_combo. Он означает, что одновременно странно размещены unwind metadata и handler target. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для placement_exec_nonexec_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'одновременно странно размещены unwind metadata и handler target', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится placement_exec_nonexec_handler_combo: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 25. Признак placement_integrity_combo

Эта глава разбирает finding placement_integrity_combo. Его короткий смысл: placement anomaly подтверждается повреждением или странностью UNWIND_INFO. Он относится к семье unwind_integrity. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для placement_integrity_combo: combined evidence. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +8. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно placement_integrity_combo становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +8. Confidence effect: corroborates placement with structural unwind damage. Mitigations: single_family_only` only if the rest of the sample stays weak. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding placement_integrity_combo. Он означает, что placement anomaly подтверждается повреждением или странностью UNWIND_INFO. Finding относится к family unwind_integrity, то есть к слою: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain

и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом

обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для placement_integrity_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'placement anomaly подтверждается повреждением или

странностью UNWIND_INFO', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится placement_integrity_combo: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 26. Признак reuse_dominance_combo

Эта глава разбирает finding reuse_dominance_combo. Его короткий смысл: массовое повторное использование unwind metadata сочетается с доминированием handler. Он относится к семье unwind_reuse. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для reuse_dominance_combo: combined evidence. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла? Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = medium, score weight = +8. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно reuse_dominance_combo становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, a handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46 строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +8. Confidence effect: corroborates centralized metadata reuse with a dominant handler target. Mitigations: single_family_only` if isolated. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding reuse_dominance_combo. Он означает, что массовое повторное использование unwind metadata сочетается с доминированием handler. Finding относится к family unwind_reuse, то есть к слою: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для reuse_dominance_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'массовое повторное использование unwind metadata сочетается с доминированием handler', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится reuse_dominance_combo: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Глава 27. Признак writable_metadata_handler_combo

Эта глава разбирает finding writable_metadata_handler_combo. Его короткий смысл: и unwind metadata, и handler target связаны с writable context. Он относится к семье metadata_placement. Семья означает не место в меню отчета, а слой рассуждения: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Начнем с простого образа. Представь, что PE-файл - это технический паспорт здания. Секции - это этажи, адреса - номера комнат, таблица RUNTIME_FUNCTION - список помещений, где может понадобиться план эвакуации, а UNWIND_INFO - сама инструкция, как безопасно выйти из функции при исключении. Finding появляется тогда, когда паспорт противоречит сам себе или инструкция лежит там, где ее не ожидают найти.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
PE	Portable Executable, формат EXE и DLL в Windows. Он содержит заголовки, секции, таблицы импорта, ресурсы и служебные директории.
секция	область PE-файла с именем, виртуальным адресом, размером и правами. Примеры: .text, .pdata, .xdata, .rdata.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
RUNTIME_FUNCTION	запись таблицы .pdata: начало функции, конец функции и адрес UNWIND_INFO.
UNWIND_INFO	структура, которая объясняет Windows, как раскрутить стек функции при исключении или unwind.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
heuristic	объяснимое инженерное правило. Оно не доказывает вредоносность, но показывает, где metadata выглядит необычно.

1. Что именно проверяет правило

Тип проверки для writable_metadata_handler_combo: combined evidence. Это важно, потому что разные признаки относятся к разным уровням. Если правило проверяет PE layout, оно работает до разбора функций. Если правило проверяет UNWIND_INFO, оно требует, чтобы адрес metadata уже был найден и переведен в bytes. Если правило относится к placement, оно сравнивает адрес metadata с именем секции и правами R/W/X.

Именно поэтому нельзя читать отчет как одну плоскую таблицу ошибок. В нем есть причинная лестница. Сначала analyzer спрашивает: можно ли доверять структуре файла?

Потом: можно ли прочитать таблицу функций? Потом: можно ли разобрать unwind metadata? И только после этого: выглядит ли расположение metadata обычным для компилятора или необычным для ручной/защитной/обфусцированной схемы?

Для текущего признака правило матрицы говорит: severity rule = high, score weight = +10. Эти числа не надо воспринимать как математический закон. Они нужны для triage: если у тебя сто файлов, score помогает выбрать, что смотреть первым. Но окончательный вывод делает человек, который смотрит affected_count, representative_rva и связанный контекст.

2. Почему это вообще считается аномалией

Windows x64 exception handling устроен строго. Когда возникает исключение или нужен unwind стека, система должна быстро найти запись функции по адресу инструкции. Для этого ей нужна корректная, сортированная и адресно надежная metadata. Если metadata повреждена или лежит в неожиданном месте, отладчик, decompiler и сам runtime могут получить неполную или ложную картину.

Конкретно writable_metadata_handler_combo становится подозрительным не потому, что слово 'подозрительно' красиво звучит, а потому что оно нарушает ожидаемый контракт. Нормальный compiler output обычно кладет code в .text, runtime function table в .pdata, unwind data в .xdata или read-only data, а handler target - в executable code. Чем дальше sample уходит от этой схемы, тем больше вопросов должен задать аналитик.

Но эвристика не равна приговору. Иногда странность объясняется JIT, protectors, sanitizer, hand-written assembly или особенностью toolchain. Поэтому проект не говорит 'это malware'. Он говорит: 'вот наблюдаемый факт, вот его адрес, вот сколько объектов затронуто, вот почему его стоит проверить руками'.

Важно: На защите не говори: 'finding доказал вредоносность'. Говори: 'finding поднял объяснимый сигнал, который требует ручной проверки и может быть усилен dynamic evidence'.

3. Как это выглядит в отчете

В HTML этот признак появляется в блоке Static findings только если он сработал. Там есть Code, Family, Severity, Affected, RVA, Section и Message. Если признак не сработал, его нет в HTML, потому что иначе отчет превратился бы в огромную таблицу нулей. Полная картина, включая нули, живет в JSON.

В JSON смотри объект heuristic_analysis. Поле findings содержит только сработавшие признаки. Но рядом есть counters: rf_oob, unwind_parse_fail, unwind_writable_section и другие. Поэтому JSON отвечает на два вопроса: какие findings появились и какие проверки прошли с нулевым результатом.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.heuristic_analysis | Format-List
```

Если finding относится к конкретным функциям или unwind records, он может отражаться и в Runtime functions table. Например, если все 46 функций указывают на metadata в секции .weird, то per-row колонка Anomalies / error честно повторит один и тот же placement warning во всех 46

строках. Это не 46 разных видов аномалий, а один массовый pattern.

4. Как проверять руками

Ручная проверка начинается с representative_rva. Это не магический адрес и не единственное место проблемы. Это удобная стартовая точка, которую analyzer выбирает, чтобы аналитик мог быстро перейти к первому характерному примеру. Дальше нужно смотреть весь affected set.

Открой HTML, найди Static findings, выпиши code и representative RVA. Затем открой PE viewer или Ghidra. Если это RVA, прибавь ImageBase для адреса в Ghidra: GhidraAddress = ImageBase + RVA. Если ImageBase равен 0x140000000, то RVA 0x6000 обычно будет 0x140006000.

Для layout findings проверяй section table. Для function table findings проверяй Begin/End/Unwind RVA. Для unwind parser findings открывай bytes UNWIND_INFO. Для placement findings смотри имя секции и permissions. Для handler findings переходи к handler RVA и смотри, является ли это настоящим кодом.

```
.\build\debug\eh_unwind_analyzer.exe "target.exe" --heuristics --json reports\target.json
--html reports\target.html
Start-Process ".\reports\target.html"
.\build\debug\eh_unwind_analyzer.exe "target.exe" --ghidra-csv reports\target_ghidra.csv
--heuristics
```

5. Как объяснить связь со score и confidence

Score weight для этого finding: +10. Confidence effect: writable unwind metadata and writable handler targets reinforce each other. Mitigations: none. Разница между score и confidence принципиальна. Score говорит, насколько много подозрительных правил сработало по весам. Confidence говорит, насколько устойчивой выглядит интерпретация.

Если finding один, confidence может быть снижен weakening rule. Если findings несколько и они принадлежат разным families, confidence растет. Если static finding подтвержден dynamic event, final assessment становится сильнее. Поэтому хороший разбор всегда связывает code, family, severity, affected_count, score и dynamic context.

6. Практический пример формулировки

Полная формулировка для отчета может звучать так: 'В binary сработал finding writable_metadata_handler_combo. Он означает, что и unwind metadata, и handler target связаны с writable context. Finding относится к family metadata_placement, то есть к слою: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code. Проверку надо начинать с representative RVA, а затем смотреть affected count и контекст в Runtime functions table'.

Если преподаватель спросит, почему этого недостаточно для окончательного вывода, ответ простой: analyzer работает как объяснимый triage-инструмент. Он показывает, где metadata не похожа на обычную compiler layout, но финальный вывод требует сравнения с кодом, toolchain и runtime behavior.

7. Контрольные вопросы

7. Углубленный учебный разбор

Связь с форматом файла

Этот признак надо начинать не с подозрения, а с формата. Формат задает обещание: где лежат данные, как они адресуются, какие размеры допустимы и какие связи между структурами ожидаются. Если finding нарушает это обещание, analyzer показывает не мнение, а конкретное расхождение между заявленной структурой и проверяемым контрактом.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с адресами

Любая ошибка в этой области в итоге сводится к адресу. Адрес может быть raw offset в файле, RVA внутри образа или VA в памяти процесса. Если человек не различает эти три уровня, он легко открывает не тот участок в Ghidra и получает ложное ощущение, что отчет ошибся.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с доверием к metadata

Unwind metadata нужна не ради красоты отчета. Она участвует в восстановлении стека, понимании функции и поиске обработчика. Когда metadata странная, reverse engineer должен снижать доверие к автоматической разметке и проверять руками.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет

найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с нормальным compiler output

Нормальный компилятор обычно производит предсказуемую картину: код в исполняемой секции, read-only metadata в data-секциях, таблицы без пересечений, handler в коде. Finding важен именно как отклонение от такой картины, а не как самостоятельное обвинение.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ложными срабатываниями

У каждого сильного правила есть добросовестные исключения. JIT, runtime, protector, sanitizer или hand-written assembly могут давать необычную картину. Поэтому проект хранит weakening rules и manual validation steps, а не выдает безусловный malware verdict.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом HTML

HTML нужен, чтобы человек быстро увидел суть. Он не обязан показывать все нулевые counters, но обязан показать сработавшие findings и дать путь к функции, секции или representative RVA. Поэтому Static findings и Runtime functions надо читать вместе.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с отчетом JSON

JSON нужен, чтобы доказать полноту проверки. Если преподаватель спросит, где остальные признаки, ответ: они находятся как счетчики в heuristic_analysis, а список возможных rules документирован в evidence_matrix.md и в этом учебнике.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с Ghidra

Ghidra превращает адреса и ranges в визуальную картину. Importer не доказывает finding сам по себе; он помогает перейти от строки отчета к функции, комментарию, bookmark и disassembly. Это мост между автоматикой и ручной проверкой.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с dynamic mode

Dynamic mode отвечает на вопрос поведения: какие API вызваны, какие handlers зарегистрированы, были ли exceptions и hits. Static finding отвечает на вопрос структуры файла. Сильный case появляется тогда, когда оба слоя рассказывают совместимую историю.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с защитой ВКР

На защите важно показать не только итог, но и путь рассуждения. Сначала объясняется термин, затем структура Windows, затем конкретное поле отчета, затем ручная проверка. Такой ответ показывает понимание, а не заученную команду.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с безопасностью запуска

Static mode можно выполнять без запуска target, поэтому он подходит для первичного triage. Dynamic mode запускает файл и требует контролируемой среды. Это различие важно проговорить, чтобы не выглядело так, будто анализатор всегда безопасно исполняет подозрительный код.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

Связь с ограничениями

Finding покрывает только свой слой. Он не видит сеть, криптографию, persistence, injection и другие области malware behavior, если они не проявляются через EH/Unwind metadata. Ограничение не ослабляет проект, а делает его честным.

Для writable_metadata_handler_combo это означает следующее: сначала надо назвать слой combined evidence, затем показать факт 'и unwind metadata, и handler target связаны с writable context', затем объяснить, какой объект затронут, и только потом обсуждать score. Такой порядок заставляет рассуждение идти от наблюдаемого факта к выводу, а не наоборот.

Если читатель совсем новичок, полезно повторить цепочку словами: файл содержит байты; байты по формату PE образуют секции; секции дают адресный контекст; контекст позволяет найти таблицу функций; таблица функций ведет к unwind metadata; metadata проверяется правилом; правило превращается в finding; finding проверяется руками.

8. Контрольные вопросы

- К какому слою относится writable_metadata_handler_combo: PE layout, RUNTIME_FUNCTION, UNWIND_INFO, placement, reuse или handler?
- Что именно считается affected object для этого finding?
- Где это видно в HTML, а где в JSON?
- Какой address надо открыть в Ghidra и как перевести RVA в VA?
- Какая benign-гипотеза могла бы объяснить такой сигнал?
- Какие dynamic events могли бы усилить или ослабить интерпретацию?

Часть IV. Практика на реальных отчетах проекта

combo_28: почему два findings затрагивают 46 функций

В combo_28 все 46 RUNTIME_FUNCTION entries имеют UNWIND_INFO в секции .weird. Сама структура UNWIND_INFO читается корректно, поэтому Decode anomalies равны 0. Но placement необычный: секция не .xdata/.rdata/.pdata и она writable. Поэтому Heuristic findings равен 2.

В Runtime functions table строка unwind section unusual; unwind section writable повторяется для каждой функции. Это нормально: каждая функция ссылается на metadata, которая лежит в одном и том же подозрительном контексте.

Dynamic validation дополнительно показывает RtlAddFunctionTable и RtlInstallFunctionTableCallback. Это runtime evidence о работе с function tables, но оно не превращает каждую строку в отдельную новую ошибку.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
HTML	самодостаточный отчет для человека: открывается браузером и показывает карточки, таблицы и детали.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.
Ghidra	инструмент reverse engineering. Здесь она используется для визуальной проверки функций, адресов и комментариев.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.

static_03: почему parse failure полезен для Ghidra

static_03 демонстрирует другой тип проблемы: одна запись указывает на UnwindInfoRVA, который не маппится в file offset. Здесь проблема не в необычном placement, а в том, что parser не может получить корректные bytes UNWIND_INFO.

Для такого случая Ghidra CSV особенно полезен. Importer может поставить warning bookmark на проблемную функцию, чтобы аналитик не искал ее вручную в большом списке.

На защите этот пример хорошо показывает разницу между 'metadata лежит странно, но читается' и 'metadata невозможно разобрать'. Это разные families и разные вопросы ручной проверки.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
HTML	самодостаточный отчет для человека: открывается браузером и показывает карточки, таблицы и детали.
JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.
Ghidra	инструмент reverse engineering. Здесь она используется для визуальной проверки функций, адресов и комментариев.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.

Как строить объяснение результата

Начни с facts: сколько Runtime functions, какой score, сколько Heuristic findings, какие code, какой affected count. Затем объясни, какой слой нарушен: table_integrity, unwind_integrity, metadata_placement, reuse или handler_dominance.

После этого покажи representative RVA и section. Если есть dynamic report, покажи events. Не прыгай сразу к verdict. Хороший аналитик строит мост от байтов к выводу.

Финальная фраза должна быть проверяемой: 'мы видим такие-то fields отчета, они означают такой-то низкоуровневый факт, ручная проверка делается по такому адресу'.

Словарь этой темы

Термины ниже не надо заучивать отдельно от проекта. Смысл в том, чтобы видеть, как одно понятие переходит в другое: байты образуют PE, PE задает секции, секции дают адресный контекст, а адресный контекст нужен, чтобы проверить unwind metadata.

Термин	Объяснение
HTML	самодостаточный отчет для человека: открывается браузером и показывает карточки, таблицы и детали.

JSON	машинно-читаемый отчет со всеми счетчиками, findings, functions и dynamic events.
Ghidra	инструмент reverse engineering. Здесь она используется для визуальной проверки функций, адресов и комментариев.
RVA	Relative Virtual Address, относительный виртуальный адрес. Это адрес внутри образа PE без прибавления ImageBase.
finding	структурированный признак в отчете: code, family, severity, message, affected_count и representative_rva.
dynamic analysis	анализ с запуском target под Windows Debug API. Analyzer фиксирует runtime events.
evidence	наблюдаемое основание для вывода: факт из PE, parser result, finding или runtime event.

Часть V. Строгий технический слой по 27 признакам

Следующий раздел включен как строгий технический слой. До этого книга объясняла материал простым языком; теперь тот же проект фиксируется более формально, ближе к документации и защите.

Том III. Полная система 27 признаков: основания, связи и ручная проверка

Что именно означает число 27

В текущей версии `heuristics.c` существует 27 возможных публичных finding codes. Это не 27 полей PE и не 27 типов атак. Это 27 именованных результатов, которые система может включить в массив:

```
"findings": []
```

Один finding может агрегировать множество однотипных нарушений. Например, `rf_oob` создаётся один раз, даже если за границы image выходят десять `RUNTIME_FUNCTION`.

Число 27 получено декомпозицией технических слоёв:

```
6 findings PE layout
7 findings RUNTIME_FUNCTION
2 findings UNWIND_INFO integrity
3 findings reuse ■ handler statistics
5 findings placement ■ permissions
4 combined findings
---
27 ■■■■■■■■■ finding codes
```

Внутри `unwind_anomalies` дополнительно различаются 12 причин `UA_*`. Поэтому глубина проверки больше, чем простое число публичных кодов, но выдавать их за 39 независимых признаков было бы неправильно.

Критерии включения признака

Finding включён в текущую систему, если выполняются следующие условия.

1. Он проверяет отдельный технический инвариант или отдельный класс контекстной аномалии. 2. Его можно вычислить детерминированно. 3. Его можно локализовать или количественно измерить. 4. Для него существует понятный manual validation workflow. 5. Его можно закрепить unit или regression test. 6. Он не дублирует полностью другой finding. 7. Для него можно назвать границы и benign alternatives.

Эти критерии важнее самого количества.

Группа I. Шесть признаков PE layout

1. `pe_section_raw_oob`

Проверяемый объект. Поля `PointerToRawData` и `SizeOfRawData` section header.

Инвариант.

```
PointerToRawData <= file_size
SizeOfRawData <= file_size - PointerToRawData
```

Что означает нарушение. Section объявляет байты, которых нет во входном файле. Любая попытка читать полный raw range без bounds check вышла бы за buffer.

Почему признак важен для EH/unwind. `.pdata` или `.xdata` может находиться в такой section. Тогда RVA формально принадлежит image, но фактические байты metadata отсутствуют.

Вес и severity.

```
weight 25
severity high
family table_integrity
```

Высокий вес связан с прямым нарушением физической доступности данных.

Benign alternatives. Обрезанная передача, повреждение архива, неверный dump memory image, нестандартный инструмент создания файла. Они объясняют происхождение, но не делают диапазон безопасным.

Ручная проверка. Сравнить сумму полей с фактическим размером файла, проверить overlay и определить, является ли объект обычным disk PE или неполным memory dump.

Что не обнаруживается. Raw range может быть доступным, но содержать семантически неверные данные.

2. `pe_section_virtual_oob`

Проверяемый объект. `VirtualAddress`, virtual span section и `SizeOfImage`.

Инвариант.

```
VirtualAddress + virtual_span <= SizeOfImage
```

Проект вычисляет virtual span с учётом `VirtualSize` и `SizeOfRawData`, поскольку реальный mapping policy должен учитывать как объявленный размер в памяти, так и доступный raw content.

Что означает нарушение. Section выходит за объявленную границу image. RVA внутри её хвоста нельзя уверенно считать принадлежащим загружаемому образу.

Связь с EH/unwind. `BeginAddress`, `EndAddress`, `UnwindInfoRVA` и `HandlerRVA` являются image-relative. Ошибка `SizeOfImage` или section span разрушает основу их проверки.

Вес и severity.

```
weight 20
severity high
```

Вес немного ниже raw OOB, потому что фактическое loader behavior может зависеть от дополнительных правил выравнивания, но модель image уже несогласованна.

Ручная проверка. Сопоставить `SizeOfImage`, `SectionAlignment`, последний section end и результат загрузки Windows loader в изолированной среде.

Граница. Признак не доказывает, что Windows обязательно загрузит файл или откажет: analyzer проверяет собственный строгий parser contract.

3. `pe_section_raw_overlap`

Проверяемый объект. Пары raw ranges sections.

Для sections A и B:

```
A = [A.raw_begin, A.raw_end)
B = [B.raw_begin, B.raw_end)
```

Overlap существует, если:

```
A.raw_begin < B.raw_end
and
B.raw_begin < A.raw_end
```

Что означает нарушение. Одни байты файла одновременно объявлены частью двух sections.

Почему это важно. Разные parser, loader и inspection tools могут интерпретировать такие bytes по-разному. Атакующий может использовать неоднозначность для parser differential.

Вес и severity.

```
weight 15
severity medium
```

Overlap значим, но некоторые упаковщики и специально сформированные images могут использовать нестандартные layouts.

Ручная проверка. Выписать обе пары raw boundaries, определить реальную область пересечения и сравнить отображение в нескольких независимых tools.

Граница. Пересечение не доказывает, что именно EH metadata была модифицирована.

4. `pe_section_virtual_overlap`

Проверяемый объект. Virtual ranges sections.

Что означает нарушение. Один RVA попадает более чем в одну section по объявленным spans.

Почему severity выше raw overlap. EH/unwind адреса выражены в RVA. Если RVA имеет две возможные section identities, неоднозначными становятся permissions, имя section и mapping в file offset.

Вес и severity.

```
weight 20
severity high
```

Возможное вмешательство. Handler может быть представлен одним parser как исполняемый, а другим как неисполняемый; metadata может скрываться в перекрывающихся mappings.

Ручная проверка. Построить отсортированный список virtual intervals, определить все sections для спорного RVA и проверить фактическое memory map после загрузки.

Граница. Проект сообщает структурную неоднозначность, но не моделирует все частные loader decisions.

5. `pe_section_misaligned`

Проверяемый объект. Соответствие section fields объявленным `FileAlignment` и `SectionAlignment`.

Типовые условия:

```
PointerToRawData mod FileAlignment = 0
VirtualAddress mod SectionAlignment = 0
```

с учётом допустимых специальных случаев формата.

Что означает нарушение. Layout не соответствует собственной alignment policy optional header.

Почему признак medium. Alignment errors могут появляться в custom, handcrafted или very-small-alignment images. Не каждое отклонение приводит к неправильному чтению EH metadata.

Вес и severity.

```
weight 8
severity medium
```

Ручная проверка. Проверить значения alignments, кратность offsets, специальные ограничения PE и фактическую загрузку.

Граница. Это общий PE signal. Он не является специфичным для `.pdata`.

6. `pe_exception_directory_oob`

Проверяемый объект. Data directory с индексом exception table.

Инвариант. Полный диапазон:

```
[ExceptionRVA, ExceptionRVA + ExceptionSize)
```

должен принадлежать image и быть отображаемым в доступные file bytes для статического разбора.

Что означает нарушение. Analyzer не может безопасно получить полный массив `RUNTIME_FUNCTION`.

Почему это прямой EH/unwind признак. Это корневая directory, из которой начинается весь статический pipeline.

Вес и severity.

```
weight 25
severity high
```

Ручная проверка. Проверить directory entry, section mapping, кратность размера 12 байтам и возможное усечение файла.

Граница. Пустая directory и directory OOB различаются. Пустота может быть допустима для leaf-only image; OOB означает недоступный объявленный диапазон.

Группа II. Семь признаков `RUNTIME_FUNCTION`

7. `no_runtime_functions`

Условие.

```
runtime_function_count == 0
and image has executable content
```

Если executable sections отсутствуют, finding не создаётся.

Почему отсутствие не всегда ошибка. Leaf function, не изменяющая nonvolatile state и не выделяющая stack frame, может раскручиваться без function table entry. Теоретически небольшой image может состоять только из таких функций.

Почему сигнал всё же нужен. Для крупного x64 image с вызовами, stack allocations и nonleaf functions полное отсутствие `.pdata` требует объяснения.

Вес и severity.

```
weight 10
severity medium
```

Ручная проверка. Дизассемблировать executable sections, найти `call`, изменения `RSP`, сохранения nonvolatile registers и определить, действительно ли весь код leaf-only.

Граница. Проект пока не выполняет эту disassembly-проверку автоматически.

8. `invalid_ranges`

Инвариант.

```
EndAddress > BeginAddress
```

Что означает нарушение. Диапазон пустой или направлен назад. Он не может однозначно описывать непустой фрагмент кода.

Threshold.

```
<= 1% affected: weight 10, medium
> 1% affected: weight 20, high
```

Почему используется доля. Одна повреждённая entry в таблице из десятков тысяч отличается по масштабу от массового нарушения. При этом даже единичная entry сохраняется как finding.

Ручная проверка. Найти entry по representative RVA, проверить endian decode, соседние records и исходные 12 bytes.

Возможное вмешательство. Изменение `EndAddress` может скрывать часть function coverage или ломать поиск текущего frame.

Граница. Корректный диапазон ещё не означает, что он совпадает с фактической функцией.

9. `rf_oob`

Инвариант.

```
BeginAddress < SizeOfImage
EndAddress <= SizeOfImage
```

Дополнительно диапазон должен соответствовать доступной executable области при ручной интерпретации.

Что означает нарушение. Function entry описывает код вне объявленного image.

Threshold и веса.

```
<= 1%: +10, medium
> 1%: +25, high
```

Почему высокий вариант имеет вес 25. Массовые out-of-image ranges разрушают function table как индекс текущего instruction pointer.

Ручная проверка. Проверить `SizeOfImage`, section ends, ASLR-independent RVA и отсутствие ошибочного смешения RVA с VA.

Граница. Внутри image range может всё равно указывать на данные, а не на код; это отдельный возможный будущий признак.

10. `unwind_rva_oob`

Проверяемый объект. Третье поле `RUNTIME_FUNCTION`.

Инвариант. `UnwindInfoRVA` должен указывать на доступную и достаточно длинную запись внутри image/file mapping.

Что означает нарушение. Function range существует, но описание обратного восстановления контекста отсутствует по указанному адресу.

Threshold и веса.

```
<= 1%: +10, medium
> 1%: +25, high
```

Почему это один из основных признаков вмешательства. Изменение одного 32-bit RVA способно перенаправить decoder на чужие данные, вызвать неправильный unwind или скрыть handler relation.

Ручная проверка. Выполнить `pe_rva_to_fileoff`, проверить section, alignment и первые четыре header bytes.

Граница. Доступный RVA может вести к синтаксически корректной, но чужой записи. Такой случай частично покрывается reuse/statistics, но требует code-to-metadata correspondence.

11. `rf_unsorted`

Нормативное ожидание. Function table entries должны быть отсортированы по `BeginAddress`.

Как считается. Во время исходного прохода проект фиксирует inversions: следующий `BeginAddress` меньше предыдущего.

Почему порядок важен. Runtime lookup использует упорядоченную таблицу. Нарушение может изменить результат поиска или сделать его implementation-dependent.

Вес и severity.

```
weight 15
severity medium
```

Почему не high автоматически. Analyzer может скопировать entries, отсортировать их и продолжить проверку overlaps. Наличие inversion не всегда означает повреждение самих ranges.

Ручная проверка. Сравнить исходный порядок `.pdata` с сортировкой, проверить, является ли directory цельной таблицей или смешением данных.

Граница. Сортировка для анализа не исправляет input и не гарантирует runtime acceptance.

12. `rf_overlaps`

Проверяемый объект. Отсортированные function intervals.

Для предыдущего максимального конца `max_end`:

```
current.BeginAddress < max_end
```

означает overlap.

Что означает нарушение. Один instruction RVA покрывается несколькими `RUNTIME_FUNCTION`.

Почему это сильный сигнал. Function table должна давать однозначную запись для virtual unwind. Overlap нарушает однозначность выбора metadata.

Вес и severity.

```
weight 25
severity high
```

Ручная проверка. Определить обе entries, проверить chained fragments и убедиться, что overlap не является ошибкой собственного определения interval.

Граница. Раздельные noncontiguous fragments допустимы через chained info, но они не должны произвольно перекрывать один instruction range.

13. `duplicate_begin`

Условие.

```
entry[i].BeginAddress == entry[i-1].BeginAddress
```

после сортировки.

Что означает нарушение. Две entries претендуют на один начальный адрес.

Отличие от overlap. Duplicate begin является более конкретной формой неоднозначности. Он может сопровождаться overlap, но сохраняется отдельно для диагностики причины.

Вес и severity.

```
weight 10
severity medium
```

Почему вес не суммируется как два независимых доказательства. Findings могут иметь общую первопричину. Поэтому analyst должен смотреть на family и raw entries, а не считать каждый code независимым экспериментом.

Ручная проверка. Сравнить `EndAddress` и `UnwindInfoRVA` обеих entries.

Граница. Finding не определяет, какая из дубликатных entries является правильной.

Группа III. Два признака `UNWIND_INFO` integrity

14. `unwind_parse_fail`

Источник. `unwind_parse` вернул `false` для одной или нескольких entries.

Parse failure означает, что decoder не смог безопасно завершить структурное чтение. Причинами могут быть:

- неизвестная version;

- усечённый header;

- недостающий operand;

- невозможный tail;

- chain read failure;

- выход mapping за доступный buffer.

Threshold и веса.

```
<= 1%: +10, medium
> 1%: +25, high
```

Почему failure отделён от anomaly. Некоторые anomalies можно отметить и продолжить parsing. Failure означает, что итоговая `UnwindSummary` не может считаться полностью полученной.

Ручная проверка. Использовать `parse_error`, исходный `UnwindInfoRVA`, hex dump и официальное описание конкретной version.

Критическая оговорка. Корректная новая version, которую parser ещё не поддерживает, также создаст parse failure. Это compatibility gap, а не повреждение файла.

15. `unwind_anomalies`

Источник. `UnwindSummary.anomalies != 0`.

Что агрегируется. Двенадцать `UA_*` flags.

Threshold.

```
<= 5% affected records: +10, medium
> 5% affected records: +20, high
```

Почему threshold выше, чем для OOB. Некоторые operand sanity и ordering отклонения могут зависеть от редких compiler patterns. Для массового нарушения вероятность системной проблемы выше.

Почему один finding, а не двенадцать. Несколько flags могут возникнуть из одного повреждённого header. Отдельное начисление веса за каждый bit искусственно увеличивало бы score.

Ручная проверка. Расшифровать short names из report:

```
ver, trunc, op, arg, off, chain_loop, chain_depth,
handler_oob, order, operand_sanity, alignment, chain_invalid
```

и проверить конкретные bytes.

Граница. Finding сообщает структурную проблему metadata, но не устанавливает её происхождение.

Группа IV. Три статистических признака

16. `unwind_reuse_moderate`

Метрика.

```
unwind_reuse_ratio =
max_count_for_one_UnwindInfoRVA / total_runtime_functions
```

Candidate condition.

```
ratio > 0.10
total > 200
```

Дополнительное условие. Должен существовать другой static signal. Умеренный reuse без corroboration подавляется.

Вес и severity.

```
weight 10
severity medium
```

Почему reuse может быть нормальным. Одинаковые prologs, shared metadata, compiler folding и runtime helpers могут законно уменьшать размер таблиц.

Почему признак всё же полезен. В сочетании с placement, integrity или handler dominance reuse может указывать на централизованную схему перенаправления.

Ручная проверка. Найти dominant `UnwindInfoRVA`, перечислить функции, проверить одинаковы ли их реальные prologs и не используется ли chained info.

17. `unwind_reuse_strong`

Условие.

```
ratio > 0.30
total > 200
```

Вес и severity.

```
weight 20
severity high
```

Почему нужен минимальный размер. В таблице из трёх entries две одинаковые дают 66.7%, но статистически это слабое основание. Условие `total > 200` исключает такие малые

samples.

Почему threshold не является универсальной нормой. Он является консервативным engineering default и должен проверяться на расширенном corpus разных toolchains.

Ручная проверка. Сравнить dominant record с распределением function sizes, compiler/runtime identity и фактическими prologs.

Граница. Сильный reuse не доказывает обфускацию.

18. `handler_dominance`

Метрика.

```
handler_reuse_ratio =
max_count_for_one_HandlerRVA / handler_count
```

Candidate condition.

```
handler_count > 50
ratio > 0.80
```

Контекстное подавление. Если dominant handler находится в executable code-like section и других static anomalies нет, finding не создаётся.

Почему это необходимо. `__C_specific_handler`, `__CxxFrameHandler3/4` и другие runtime handlers могут законно обслуживать множество функций.

Вес и severity.

```
weight 15
severity medium
```

Когда finding становится значимым. При сочетании с writable/non-executable placement, integrity errors или необычным reuse.

Ручная проверка. Идентифицировать handler, проверить imports/symbols, section permissions, xrefs, language-specific data и runtime hits.

Граница. Статический dominant handler может никогда не исполняться на исследованном path.

Группа V. Пять placement-признаков

19. `unwind_in_exec_section`

Условие. Section, содержащая `UnwindInfoRVA`, имеет `IMAGE_SCN_MEM_EXECUTE`.

Ожидаемая модель. Unwind metadata является данными и обычно размещается в read-only `.xdata`, `.rdata` или related section.

Почему executable placement подозрителен. Bytes могут одновременно интерпретироваться как code и metadata; это повышает риск parser differential, сокрытия и self-referential layouts.

Вес и severity.

```
weight 20
severity high
```

Ручная проверка. Проверить section flags, дизассемблирует ли инструмент эту область как code, есть ли control-flow edges внутрь metadata.

Граница. Нестандартный linker может объединить code и read-only data. Finding требует объяснения, но не является доказательством атаки.

20. `unwind_unusual_section`

Условие. Section name не относится к разрешённым xdata/rdata/pdata-like именам и не распознаётся как plausibly unwind-named.

Threshold.

```
count <= total / 20: +8, medium
count > total / 20: +15, high
```

Почему имя является слабым источником. PE section name не определяет семантику и может быть произвольным. Поэтому permissions сильнее имени.

Зачем правило существует. Необычное размещение помогает analyst быстро найти custom linker/protector regions и сопоставить их с другими signals.

Weakening. Placement-only и single-family penalties могут снизить confidence.

Ручная проверка. Проверить characteristics, entropy, соседние data, compiler identity и mapping всех affected records.

Implementation nuance. При `total < 20` integer division даёт `total / 20 == 0`, поэтому любое положительное count попадает в high branch. Это кандидат для будущей calibration.

21. `unwind_writable_section`

Условие. Section с `UNWIND_INFO` имеет `IMAGE_SCN_MEM_WRITE`.

Почему это сильнее необычного имени. Writable permission является функциональным свойством memory mapping. После загрузки metadata потенциально может изменяться без изменения disk file.

Threshold и веса.

```
■■ 5%: +10, medium
■■■■■ 5%: +18, high
```

Возможные benign причины. JIT infrastructure, custom runtime, post-load fixups, объединённая data section.

Ручная проверка. Определить, изменяются ли bytes во время исполнения; сравнить disk и memory copies; проверить calls к APIs управления memory protection.

Граница. Сам permission не доказывает фактическую модификацию.

22. `handler_nonexec_section`

Условие. `HandlerRVA` не принадлежит известной executable section.

Почему это сильный structural signal. Handler должен быть передан управлению как код. Target в non-executable data противоречит обычной модели исполнения.

Вес и severity.

```
weight 15
severity high
```

Возможные объяснения. Неверный RVA, повреждённая metadata, section table anomaly, runtime protection transition, custom loader.

Ручная проверка. Проверить mapping после загрузки, DEP permissions, возможное изменение protection и фактическое значение handler в memory.

Граница. Static section flags не всегда равны текущим page protections после `VirtualProtect`.

23. `handler_writable_section`

Условие. Handler target находится в section с write permission.

Почему это важно. Writable executable code облегчает runtime modification и затрудняет утверждение, что disk bytes равны исполнявшимся bytes.

Вес и severity.

```
weight 10
severity medium
```

Почему не high автоматически. Некоторые runtime-generated или unpacked regions законно проходят состояние writable+executable.

Ручная проверка. Проверить W^X policy, фактические page protections, изменения bytes и момент установки handler.

Граница. Section-level permission грубее page-level runtime protection.

Группа VI. Четыре combined findings

24. `placement_exec_nonexec_handler_combo`

Условие.

```
unwind_in_exec_section exists
and handler_nonexec_section exists
```

Логика. Metadata размещена там, где ожидается code, а handler указывает туда, где code не ожидается. Два направления нарушения усиливают друг друга.

Вес и severity.

```
weight 10
severity high
```

Почему это отдельный finding. Он выражает не новую raw observation, а policy о значимости совместного появления.

Ручная проверка. Проверить обе стороны по конкретным RVA и исключить ошибку section mapping.

25. `placement_integrity_combo`

Условие.

```
(unwind in executable section or handler non-executable)
and
(parse failure or unwind anomaly or unwind RVA OOB)
```

Логика. Необычное размещение подкреплено повреждением содержимого или указателя.

Вес и severity.

```
weight 8
severity medium
family unwind_integrity
```

Почему вес умеренный. Исходные findings уже внесли основной вклад. Combo добавляет ограниченный corroboration bonus.

Граница. Сигналы могут иметь одну первопричину, поэтому их нельзя интерпретировать как независимые вероятности.

26. `reuse_dominance_combo`

Условие.

```
unwind_reuse_ratio > 0.10
and handler_dominance finding exists
```

Логика. Централизация происходит одновременно на уровне unwind records и handler targets.

Вес и severity.

```
weight 8
severity medium
```

Возможная benign причина. Общий language runtime handler плюс compiler-generated shared unwind descriptions.

Ручная проверка. Проверить, соответствует ли dominant handler известному runtime и согласованы ли реальные prologs с общей metadata.

27. `writable_metadata_handler_combo`

Условие.

```
unwind_writable_section exists
and handler_writable_section exists
```

Логика. И описание unwind, и код handler находятся в изменяемых regions. Это создаёт возможность согласованного runtime изменения обеих сторон.

Вес и severity.

```
weight 10
severity high
```

Ручная проверка. Снять memory snapshots до и после инициализации, сравнить bytes, page protections и события регистрации.

Граница. Permission создаёт возможность изменения, но не доказывает, что изменение состоялось.

Почему отдельных findings не больше

Проект не создаёт отдельные публичные codes для каждого:

- `UWOP`;
- register number;
- `UA_*` bit;
- section name;
- dynamic exception code.

Причина - контроль гранулярности. Слишком мелкие findings:

- многократно оценивают одну первопричину;
- переполняют report;
- затрудняют сравнение файлов;
- делают score зависимым от случайного количества вторичных симптомов.

Низкоуровневые детали сохраняются в `UnwindSummary`, counters, operations text и event timeline.

Почему отдельных findings не меньше

Объединение всех ошибок в один code `malformed_unwind` потеряло бы:

- слой возникновения;
- severity;
- локализацию;
- manual workflow;
- возможность threshold calibration;
- проверяемость regression tests.

Разделение на 27 codes сохраняет техническую причину результата.

Матрица покрытия классов вмешательств

Класс изменения	Основные findings	Что подтверждается
Усечение или ложные file ranges	`pe_section_raw_oob`, `pe_exception_directory_oob`	Объявленные bytes недоступны
Ложный virtual layout	`pe_section_virtual_oob`, overlaps, misalignment	Image model неоднозначна или несогласованна

| Подмена function boundaries | `invalid_ranges`, `rf_oob`, `rf_unsorted`, `rf_overlaps`, `duplicate_begin` | Function table нарушает structural contract |

| Подмена unwind pointer | `unwind_rva_oob`, `unwind_parse_fail` | Запись недоступна или не разбирается |

| Повреждение unwind encoding | `unwind_anomalies` | Нарушены version-specific rules |

| Перемещение metadata | три unwind placement findings | Section context необычен |

| Перенаправление handler | два handler placement findings | Target section не соответствует обычному коду |

| Централизация | reuse и dominance | Распределение ссылок необычно |

| Сочетание независимых слоёв | четыре combo findings | Несколько свойств взаимно подкрепляются |

Какие вмешательства 27 признаков не гарантируют обнаружить

1. Изменение машинного кода при полностью согласованной metadata. 2. Замена handler на другой допустимый executable RVA. 3. Подмена semantics внутри существующего handler без изменения placement. 4. Runtime modification после окончания observation window. 5. Динамическая регистрация через ненаблюдаемый механизм. 6. Неисполненный path. 7. Корректная новая версия metadata без поддержки parser. 8. Изменение language-specific handler data, которое не нарушает текущие structural checks. 9. Подмена файла на другой валидный файл без trusted baseline.

Как добавить новый признак правильно

Новый finding должен пройти последовательность.

1. Сформулировать технический инвариант или статистическую гипотезу. 2. Определить raw observations. 3. Назвать benign alternatives. 4. Выбрать family. 5. Обосновать severity и вес относительно существующих rules. 6. Добавить representative location и affected count. 7. Добавить unit test positive. 8. Добавить unit test benign suppression. 9. Проверить влияние на corpus и thresholds. 10. Обновить schema semantics, evidence matrix и учебный курс.

Итоговое обоснование числа 27

Технически корректный ответ:

> В проекте 27 публичных finding codes, потому что текущая модель > декомпозирована на 6 проверок PE layout, 7 проверок function table, 2 > агрегированных проверки `UNWIND_INFO`, 3 статистических pattern, 5 > placement-проверок и 4 combined rules. Число не объявляется полным числом > возможных атак. Полнота оценивается по покрываемым классам вмешательств, > traceability и тестам каждого правила. Внутри unwind decoder дополнительно > различаются 12 диагностических причин `UA_*`.

Контрольные вопросы

1. Почему `duplicate_begin` и `rf_overlaps` сохраняются отдельно? 2. Почему moderate reuse требует corroboration? 3. Почему handler dominance подавляется для обычного runtime handler? 4. Почему section name слабее section permission? 5. Почему `unwind_parse_fail` и `unwind_anomalies` разделены? 6. Почему combo finding может добавлять вес, хотя raw observations уже учтены? 7. Какие признаки являются жёсткими structural violations? 8. Какие признаки являются контекстными? 9. Какие признаки являются статистическими? 10. Как доказать, что новый finding не дублирует существующий?

Том III. Точность, валидация и проектирование строгого эксперимента

Почему вопрос «какова точность» неполон

Точность нельзя вычислить, пока не определены:

1. единица классификации; 2. положительный класс; 3. отрицательный класс; 4. ground truth; 5. threshold решения; 6. состав corpus; 7. правила учёта неоднозначных cases.

Для EH/Unwind Analyzer возможны разные задачи:

- определить, разбирается ли PE без structural failure;

- определить, существует ли конкретное нарушение;

- определить, должен ли finding быть создан;

- определить, требует ли файл ручной проверки;

- определить, присутствовало ли намеренное вмешательство;

- определить, является ли файл вредоносным.

Проект непосредственно решает первые четыре задачи. Последние две требуют дополнительного ground truth и анализа semantics.

Единица оценки

Unit of evaluation, или единица оценки, определяет, что считается одним примером.

File-level

Один PE-файл является одним примером.

Вопрос:

```
■■■■■■■ ■■ ■■■■■ ■■■■■■■■■ ■■■■■ ■■ ■■■■■ finding
```

Преимущество - простая итоговая таблица. Недостаток - файл с одной повреждённой entry и файл с тысячами повреждений учитываются одинаково.

Record-level

Одна `RUNTIME_FUNCTION` или одна `UNWIND_INFO` является примером.

Вопрос:

```
■■■■■■■■■■ ■■ ■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■
```

Это точнее для parser и per-record anomalies, но требует детальной разметки каждой записи.

Finding-level

Один потенциальный finding code для одного файла является примером.

Вопрос:

```
■■■■■■■ ■■ code X ■■■■■■■■■■■■■■■ ■ ■■■■■■■ ■■■■■■ Y
```

Так можно измерять precision и recall отдельно для каждого из 27 rules.

Event-level

Одно динамическое событие является примером.

Вопрос:

```
[redacted] handler hit
```

Требуется внешний trace или специально контролируемый target.

Ground truth

Ground truth - заранее установленная эталонная разметка, с которой сравнивается результат инструмента.

Ground truth должен отвечать:

- какое нарушение присутствует;

- где оно находится;

- почему оно считается нарушением;

- каким способом это подтверждено;

- кто выполнил разметку;

- допускается ли неоднозначный verdict.

Источники ground truth:

1. специально сконструированный sample; 2. differential comparison с независимыми parsers; 3. compiler/linker provenance; 4. ручная экспертная проверка; 5. сравнение disk/memory/baseline versions; 6. instrumented dynamic target; 7. consensus нескольких экспертов.

Synthetic sample даёт точный ground truth по внесённому изменению, но плохо представляет разнообразие реальных toolchains. Реальный corpus представляет разнообразие, но требует дорогой разметки.

Confusion matrix

Для бинарного решения используются четыре результата.

True Positive, TP

Нарушение присутствует, и инструмент его обнаружил.

False Positive, FP

Нарушение отсутствует, но инструмент создал положительный сигнал.

True Negative, TN

Нарушение отсутствует, и инструмент не создал сигнал.

False Negative, FN

Нарушение присутствует, но инструмент его не обнаружил.

Матрица:

| Ground truth | Сигнал есть | Сигнала нет |

| --- | ---: | ---: |

| Нарушение есть | TP | FN |

| Нарушения нет | FP | TN |

Без размеченных positive examples нельзя вычислить TP и FN. Без размеченных negative examples нельзя вычислить TN и FP.

Accuracy

Accuracy, или доля правильных решений:

```
accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Accuracy сильно зависит от баланса classes.

Если в corpus 990 benign и 10 anomalous files, инструмент, всегда отвечающий «аномалии нет», получит:

```
accuracy = 990 / 1000 = 99%
```

но:

```
recall = 0 / 10 = 0%
```

Поэтому overall accuracy не должна быть единственной метрикой triage tool.

Sensitivity и recall

Sensitivity, также **recall** или true positive rate:

```
recall = TP / (TP + FN)
```

Она отвечает:

```
[illegible]
```

Для проекта recall нужно измерять per finding class. Например, отдельный recall для:

- `rf_overlaps`;
- `unwind_rva_oob`;
- `UA_BAD_OPCODE`;
- writable handler placement;
- runtime `RtlAddFunctionTable` registration.

Объединение всех нарушений в одну категорию скрывает слабые классы.

Specificity

Specificity, или true negative rate:

```
specificity = TN / (TN + FP)
```

Она отвечает:

```
[illegible]
```

Для analyst-facing triage specificity важна, потому что false positives создают ручную нагрузку.

Precision

Precision, или positive predictive value:

```
precision = TP / (TP + FP)
```

Она отвечает:

```
[illegible] ground truth
```

Precision зависит от prevalence нарушений в corpus. Даже неизменный detector может иметь разную precision в исследовательском наборе с 50% anomalies и в реальном потоке с 0.1% anomalies.

Negative Predictive Value

```
NPV = TN / (TN + FN)
```

Показатель отвечает, насколько надёжен отрицательный результат в выбранном corpus. Для проекта его нельзя интерпретировать как гарантию отсутствия любого вмешательства, потому что проверяется ограниченное пространство классов.

F1

F1 является гармоническим средним precision и recall:

```
F1 = 2 * precision * recall / (precision + recall)
```

F1 не использует TN и полезна при дисбалансе classes, но также скрывает различие между precision и recall. Поэтому обе исходные метрики должны публиковаться отдельно.

False Positive Rate

```
FPR = FP / (FP + TN) = 1 - specificity
```

Если specificity равна 99%, FPR равна 1%. Для потока из миллиона benign файлов это теоретически до десяти тысяч сигналов при аналогичном распределении. Абсолютная analyst workload зависит от объёма данных.

False Negative Rate

```
FNR = FN / (FN + TP) = 1 - recall
```

Этот показатель особенно важен для жёстких structural violations, где ожидается почти детерминированное обнаружение при условии поддерживаемой версии формата.

Threshold

Проект имеет несколько thresholds:

- prevalence выше 1% для ряда integrity findings;

- anomaly prevalence выше 5%;

- reuse выше 10% или 30%;

- dominance выше 80%;

- score verdict thresholds 15, 40 и 70.

Изменение threshold меняет trade-off.

Снижение threshold:

- обычно увеличивает recall;

- обычно увеличивает false positives.

Повышение threshold:

- обычно увеличивает specificity;

- может увеличить false negatives.

Threshold нельзя выбирать только по synthetic samples, специально созданным под существующие правила.

ROC и PR curves

Если система выдаёт непрерывный score, можно изменять decision threshold и строить curve.

ROC curve показывает:

```
true positive rate ■■■■■■ false positive rate
```

PR curve показывает:

```
precision ■■■■■■ recall
```

Для редких positive classes PR curve часто информативнее ROC, потому что сильнее отражает влияние false positives.

Текущий score является суммой policy weights. Построение curve возможно технически, но имеет смысл только на независимом размеченном corpus.

Calibration

Calibration отвечает на вопрос, соответствует ли числовая оценка эмпирической частоте события.

Если система заявляет вероятность 0.8 для ста cases, приблизительно 80 из них должны подтверждаться при хорошей calibration.

Поле `suspicion = score / 100` не calibrated probability. Поэтому для него нельзя применять вероятностную интерпретацию.

Confidence interval

Результат sample является оценкой неизвестного свойства более широкой совокупности.

Если в 106 benign files нет false positives:

```
observed specificity = 106 / 106 = 100%
```

Это не доказывает universal specificity 100%. При простой биномиальной модели двусторонний 95% Wilson interval имеет нижнюю границу приблизительно 96.5%.

Wilson interval для доли:

```
center =
(p_hat + z^2/(2n)) / (1 + z^2/n)

half_width =
z / (1 + z^2/n)
* sqrt(p_hat(1-p_hat)/n + z^2/(4n^2))
```

где:

```
p_hat = x / n
z = 1.96 ■■■ ■■■■■■■■■■■■■■■■■ 95% interval
```

Даже interval не исправляет corpus bias. Если все files получены от ограниченного набора vendors и toolchains, математическая формула не делает выборку репрезентативной автоматически.

Independence assumption

Простая биномиальная модель предполагает независимые испытания.

106 files могут содержать:

- десятки DLL одной CPython distribution;

- несколько binaries одного compiler version;

- общие runtime libraries;

- похожие linker options.

Такие samples коррелированы. Эффективное количество независимых layouts меньше числа файлов. Поэтому полезно группировать результаты по:

- vendor;

- product;

- compiler;

- linker;

- architecture feature;

- presence of handlers;

- number of runtime functions.

Что доказывает corpus 106/106

Зафиксированный результат:

```
106/106 valid reports
174 795 RUNTIME_FUNCTION
0 parse failures
0 structural unwind anomalies
0 nonzero heuristic scores
```

Он подтверждает:

1. совместимость parser с выбранным benign corpus; 2. отсутствие наблюдаемых false positives по правилу «ненулевой score/finding является положительным сигналом»; 3. способность обработать большое число реальных entries; 4. стабильность outputs на этих layouts.

Он не подтверждает:

- recall на реальных вмешательствах;

- поддержку всех Windows toolchains;

- поддержку V3;

- отсутствие false positives в мировой совокупности;

- общую accuracy.

Что доказывают 6 regression samples

Два benign и четыре suspicious fixtures проверяют:

- ожидаемые paths parser;

- срабатывание конкретных известных rules;

- score/verdict contract;

- стабильность JSON.

Это **functional regression coverage**.

Оно не является оценкой external sensitivity, потому что suspicious samples сконструированы под известные conditions. Инструмент фактически проверяется на задачах, которые разработчик заранее вложил в fixtures.

Что доказывают 1000 mutations

Результат:

```
634 accepted valid reports
366 controlled rejections
0 crashes
0 hangs
0 invalid reports
```

подтверждает robustness относительно выполненных mutations:

- malformed inputs обрабатываются контролируемо;

- parser не упал;

- report contract сохранился.

Это не classification accuracy. Для большинства mutations неизвестно, какой finding должен был быть истинным.

Что доказывают 124/124 Windows reports

Результат означает, что все собранные reports прошли JSON Schema и cross-field semantic validation.

Проверяется:

- структура report;
- согласованность counters;
- status fields;
- event bounds;
- metadata;
- отсутствие нарушений schema.

Это не означает, что все возможные dynamic paths были достигнуты.

Что доказывают 4/4 focused smoke scenarios

Четыре purpose-built targets подтверждают работу selected mechanisms:

- VEH;
- VCH;
- UEF;
- runtime function-table registration.

Это проверка известных scenarios. Она не измеряет coverage реальных программ, anti-debug behavior, multi-process logic и всех способов регистрации.

Parser accuracy и detector accuracy

Эти понятия необходимо разделять.

Parser structural accuracy

Вопрос:

```
■■■■■■■■■ ■■ bytes ■■■■■■■■■■■■■ ■ ■■■■ ■ summaries
```

Методы:

- byte-level golden fixtures;
- differential comparison;
- round-trip там, где применимо;
- compiler-generated samples;
- boundary unit tests.

Heuristic detection accuracy

Вопрос:

```
[illegible] [illegible] findings [illegible] [illegible] [illegible]
```

Методы:

- per-class labeled corpus;
- precision/recall;
- threshold sweep;
- error analysis.

Runtime observation accuracy

Вопрос:

```
[illegible] [illegible] collector [illegible] [illegible] [illegible]
```

Методы:

- instrumented target ground truth;
- independent debugger/ETW trace;
- timestamps and event identities;
- repeated runs.

Нельзя объединять эти три слоя в один процент.

Строгий benchmark plan

Шаг 1. Зафиксировать taxonomy

Для каждого из 27 findings определить:

- positive condition;
- negative condition;
- ambiguous/excluded condition;
- expected location;
- expected affected count.

Шаг 2. Собрать corpus

Минимальные strata:

- MSVC разных versions;
- MinGW-w64;
- Clang/LLVM for Windows;
- system binaries;
- user applications;

- DLL, EXE и PYD;

- binaries с C, C++ EH и без handlers;

- packed/protected samples;

- handcrafted malformed samples;

- V1, V2 и V3 после реализации поддержки.

Шаг 3. Разделить development и test

```
development set -> ■■■■■■ rules ■ thresholds
test set -> ■■■■■■■■■■■■ ■■■■■■■■■ ■■■■■■■
```

Использование test set для настройки превращает его в development set.

Шаг 4. Выполнить независимую разметку

Желательно два эксперта. Расхождения фиксируются и разрешаются отдельно. Нужно хранить rationale, а не только label.

Шаг 5. Измерить per-class metrics

Для каждого finding:

- TP;

- FP;

- TN;

- FN;

- precision;

- recall;

- specificity;

- F1;

- confidence interval.

Шаг 6. Проверить thresholds

Запустить score sweep и prevalence threshold variants. Выбирать policy по заранее определённой цели, например:

```
■■■■■■■■■■■■■■■ false-positive burden
■■■■ recall ■■■■■■■■ structural violations ■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■
```

Шаг 7. Провести error analysis

Каждый FP и FN должен получить:

- root cause;

- affected toolchain;

- решение: исправить, документировать или исключить;

- regression fixture, если исправление принято.

Шаг 8. Опубликовать limitations

Benchmark должен указывать:

- даты;
- hashes;
- версии;
- состав corpus;
- exclusions;
- confidence intervals;
- known bias.

Какая метрика главная для этого проекта

Одной главной метрики нет.

Для hard structural rules приоритет:

```
■■■■■■■■ per-class recall
```

Для контекстных placement/reuse rules:

```
■■■■■■■■ precision
■ ■■■■■■■ analyst false-positive burden
```

Для benign compatibility:

```
specificity
```

Для parser robustness:

```
crash-free execution, controlled rejection, sanitizer/fuzz results
```

Для dynamic collector:

```
event recall ■■ instrumented targets
■ ■■■■■■■■■■■■■ event semantics
```

Технически корректный ответ о текущей точности

> Общая accuracy проекта пока не установлена, потому что нет достаточно > большого независимого размеченного positive corpus для всех классов > вмешательств. На выбранном benign corpus наблюдаемая file-level specificity > составила 106/106, то есть 100%; нижняя граница простого 95% Wilson interval > приблизительно 96.5%, но выборка ограничена. Шесть regression samples > подтверждают ожидаемую функциональность известных rules, 1000 mutations - > robustness, 124 Windows reports - корректность schema/semantics, а 4 smoke > scenarios - работу выбранных dynamic mechanisms. Эти результаты нельзя > складывать в один процент.

Технические источники

1. NIST Static Analysis Tool Study Methodology - определения precision, accuracy и recall при сравнении analysis tools: `https://www.nist.gov/system/files/documents/2021/03/24/CAS%202012%20Static%20Analysis%20Tool%20Study%20Methodology.pdf` 2. NIST/SEMATECH e-Handbook of Statistical Methods - confidence intervals и основы статистической оценки: `https://www.itl.nist.gov/div898/handbook/` 3. Документация проекта: `docs/evidence_matrix.md`, `docs/threshold_calibration.md`, `docs/PRODUCT_READINESS.md`.

Контрольные вопросы

1. Почему 106/106 не является общей accuracy? 2. Чем specificity отличается от precision? 3. Почему synthetic positive samples не дают внешнюю sensitivity? 4. Что измеряют mutations? 5. Зачем разделять file-level и finding-level metrics? 6. Почему один accuracy-процент опасен при class imbalance? 7. Что означает confidence interval? 8. Почему correlated binaries уменьшают effective sample diversity? 9. Как избежать leakage между development и test set? 10. Какие метрики нужны отдельно для parser, heuristics и dynamics?

Том III. `UNWIND_INFO` V3: новая модель, совместимость и план поддержки

Почему V3 требует отдельной главы

В ранних главах курс описывает `UNWIND_INFO` V1/V2. Текущий decoder проекта поддерживает именно эти версии. В мае 2026 года Microsoft опубликовала preview specification `x64 Unwind Information V3`.

V3 нельзя считать небольшим добавлением одного opcode. Она сохраняет `RUNTIME_FUNCTION` и общие `.pdata`/`.xdata` conventions, но существенно меняет внутреннюю организацию unwind record:

- вместо V1/V2 slots появляется variable-length WOD pool;
- отдельно хранятся IP offsets;
- epilogs получают явные descriptors;
- integer register encoding расширяется до 5 bits для APX registers;
- появляется `UNW_FLAG_LARGE`;
- payload имеет собственный размер в 16-bit words.

Попытка читать V3 decoder-ом V1/V2 создаст неверные boundaries, operations и tail offset. Поэтому controlled rejection текущей версии проекта безопаснее ложного успешного разбора.

Статус спецификации

Microsoft прямо обозначает V3 как preview specification. Это означает:

- структура уже документирована;
- compiler/assembler support может появляться экспериментально;
- возможны несовместимые изменения;
- production parser должен versionировать реализацию и тестовые fixtures;
- нельзя считать текущий текст неизменяемым навсегда.

На дату подготовки главы официальный источник указывает обновление 21 мая 2026 года.

Назначение V3

V3 добавляет поддержку Intel APX, включая расширенный набор general-purpose registers R16-R31. Она также предоставляет compiler больше свободы в prolog и epilog и позволяет точнее описывать операции обоих направлений.

Ключевое различие:

```
V1/V2 ■ ■■■■■■■■ ■■■■■■■■ ■■■■■■■■ prolog,
■ unwind procedure ■■■■■■■■■■■■■■■■ ■■■■■■■■ ■ ■■■■■■■■ ■■■■■■■■.

V3 ■■■■■■■ ■■■■■■ pool operation descriptors,
■■ ■■■■■■■■ ■■■■■■■■■■ prolog ■ ■■■■■■ epilog descriptors.
```

Что осталось неизменным

`.pdata` по-прежнему является отсортированным массивом 12-byte `RUNTIME_FUNCTION`:

```
offset 0: BeginAddress
offset 4: EndAddress
offset 8: UnwindInfoAddress
```

Все три поля остаются 32-bit RVA.

Следовательно, существующие findings function table остаются концептуально применимыми:

- `invalid_ranges`;
- `rf_oob`;
- `unwind_rva_oob`;
- `rf_unsorted`;
- `rf_overlaps`;
- `duplicate_begin`.

Но decoder объекта по `UnwindInfoAddress` должен выбирать implementation по version.

Header V3

Header занимает четыре bytes.

Byte	Bits	Поле
0	2:0	`Version = 3`
0	7:3	`Flags`
1	7:0	`SizeOfProlog` low byte
2	7:0	`PayloadWords`
3	4:0	`NumberOfOps`
3	7:5	`NumberOfEpilogs`

Сходство размера header с V1/V2 не означает совместимость полей.

В V1/V2 byte 2 - `CountOfCodes`, а byte 3 содержит frame register и frame offset. В V3:

```
byte 2 = PayloadWords
byte 3 = NumberOfOps + NumberOfEpilogs
```

Старый decoder истолковал бы эти значения как slots и frame declaration.

`PayloadWords`

`PayloadWords` задаёт число 16-bit words после header:

```
payload_size = PayloadWords * 2
```

Payload содержит:

1. optional large-prolog extension; 2. prolog IP offsets; 3. epilog descriptors; 4. epilog IP offsets; 5. WOD pool; 6. возможный padding до полного word.

Handler RVA или chained `RUNTIME_FUNCTION` в `PayloadWords` не входят.

Tail offset

Положение handler/chain вычисляется:

```
tail =
align_up(sizeof(UNWIND_INFO_V3) + PayloadWords * 2, 4)
```

После tail:

- при `UNW_FLAG_EHANDLER` или `UNW_FLAG_UHANDLER` находится 4-byte

handler RVA и language-specific data;

- при `UNW_FLAG_CHAININFO` находится 12-byte `RUNTIME_FUNCTION`.

Общий принцип tail сохранился, но его исходная длина определяется payload, а не количеством V1/V2 slots.

Flags V3

V3 сохраняет:

```
UNW_FLAG_EHANDLER = 0x01
UNW_FLAG_UHANDLER = 0x02
UNW_FLAG_CHAININFO = 0x04
```

Добавляется:

```
UNW_FLAG_LARGE = 0x08
```

Bit 4 зарезервирован и должен быть zero.

Decoder должен проверять:

- несовместимые combinations;

- reserved bit;

- наличие required extension;

- достаточную длину payload;

- tail bounds.

Large prolog

Без `UNW_FLAG_LARGE` размер prolog ограничен 8-bit полем.

При `UNW_FLAG_LARGE` первый byte payload является `UNWIND_INFO_LARGE_V3.SizeOfPrologHighByte`.

Полный размер:

```
SizeOfProlog16 =
(SizeOfPrologHighByte << 8) | SizeOfProlog
```

Кроме того, prolog IP offsets становятся 16-bit вместо 8-bit.

Это означает, что parser должен вычислять element width из flags до чтения массива offsets.

Prolog IP offsets

Количество элементов равно `NumberOfOps`.

Обычная форма:

```
NumberOfOps bytes
```

Large form:

```
NumberOfOps * 2 bytes
```

Первый offset относится к операции, ближайшей к function body, то есть к последней prolog instruction с unwind effect. Последний offset относится к операции, ближайшей к function entry.

Таким образом сохраняется принцип reverse execution order.

Epilog descriptors

`NumberOfEpilogs` кодируется тремя bits и имеет диапазон 0-7 для одного fragment. Если epilogs больше, specification предлагает subfragments.

Каждый `EPILOG_INFO_V3` содержит:

```
Flags 3 bits
NumberOfOps 5 bits
EpilogOffset signed 16 bits
```

При `NumberOfOps > 0` следует extended descriptor:

- `FirstOp`;

- `IpOffsetOfLastInstruction`;

- массив IP offsets.

При `NumberOfOps == 0` descriptor наследует operation description от предыдущего epilog. Parser должен запретить наследование без предыдущего полного descriptor.

Epilog offset

`EpilogOffset` является signed displacement.

Для первого descriptor:

- положительное значение считается от fragment start;

- отрицательное - от fragment tail.

Для последующих descriptors значение является delta от предыдущего epilog.

Все epilogs должны использовать одинаковое направление:

```
■■■■ ascending from start,
■■■■ descending from tail
```

Нарушение sign/order policy должно стать отдельной V3 anomaly reason или подтипом общего descriptor anomaly.

Parent fragment transfer

Epilog flag `EPILOG_INFO_PARENT_FRAGMENT_TRANSFER` сообщает, что epilog передаёт управление parent fragment, например через `JMP`, а не возвращается непосредственно caller.

Для reverse engineer это важно: control transfer нельзя автоматически интерпретировать как обычный `RET`.

Large epilog

Flag `EPILOG_INFO_LARGE` выбирает extended descriptor с 16-bit `IpOffsetOfLastInstruction` и 16-bit elements массива offsets.

Parser должен независимо учитывать:

- large prolog policy;

- large policy конкретного epilog.

Нельзя использовать один global element width для всего payload.

WOD

Winding Operation Descriptor, WOD - variable-length packed descriptor одной unwind operation.

WOD pool является последовательностью bytes без выравнивания между descriptors. Размер одного WOD определяется opcode и составляет от одного до пяти bytes.

Prolog WOD list начинается с offset zero WOD pool. Epilog использует `FirstOp`, который является byte offset, а не порядковым номером operation.

Почему variable-length decoding сложнее

В V1/V2 основная единица массива - двухбайтовый slot. Длина operation выражается числом slots.

В V3:

```
operation length = 1..5 bytes
opcode width = 3, 4, 6 ■■■ 8 bits
```

Decoder должен проверять opcode в определённом порядке. Если сначала сравнить полный byte, можно неправильно распознать компактные opcodes, которые используют только младшие bits.

Порядок dispatch WOD

Безопасная логика:

1. проверить 3-bit compact opcodes; 2. проверить 4-bit opcodes; 3. проверить 6-bit `WOD_PUSH2`; 4. проверить full-byte opcodes; 5. иначе отметить invalid opcode.

Этот порядок является частью формата, а не только implementation detail.

Основные WOD

WOD	Размер	Смысл
`WOD_SET_FPREG`	2	Установить frame register
`WOD_ALLOC_HUGE`	5	Большое unscaled выделение stack
`WOD_ALLOC_LARGE`	3	Выделение stack с scale 8
`WOD_PUSH_CANONICAL_FRAME`	2	Canonical OS/hardware frame
`WOD_PUSH`	1	Push одного integer register
`WOD_SAVE_NONVOL_FAR`	5	Far save integer register
`WOD_SAVE_NONVOL`	3	Scaled save integer register
`WOD_PUSH_CONSECUTIVE_2`	1	Push двух последовательных registers
`WOD_ALLOC_SMALL`	1	Compact stack allocation
`WOD_SAVE_XMM128_FAR`	5	Far save XMM
`WOD_SAVE_XMM128`	3	Scaled save XMM
`WOD_PUSH2`	2	APX push двух arbitrary registers

Integer register encoding

V3 compact integer operations используют 5-bit register number:

```
0..15 -> RAX..R15
16..31 -> R16..R31
```

Это одно из принципиальных APX-изменений. Текущий `UnwindSummary` имеет:

```
uint16_t saved_nonvol_mask;
```

Такой mask способен представить только 16 register numbers. Для V3 он недостаточен.

Поддержка V3 требует как минимум:

```
uint32_t saved_nonvol_mask;
```

или отдельной extensible register-set structure.

Изменение структуры влияет на:

- `unwind.h`;
- `report.c`;
- JSON schema;
- CSV;
- HTML;
- tests;
- backward compatibility.

`WOD_PUSH`

Первый byte:

```
bits 2:0 = 100b
bits 7:3 = 5-bit register
```

Operation уменьшает `RSP` на 8 при исполнении prolog. При unwind register восстанавливается из stack, а simulated `RSP` увеличивается на 8.

Parser должен проверять, допустим ли register для данной ABI role, а не только помещается ли номер в пять bits.

`WOD_PUSH2`

`WOD_PUSH2` занимает два bytes и кодирует два arbitrary 5-bit registers.

Stack effect:

```
16 bytes
```

Для summary одного `pushed_nonvol_count` достаточно для количества, но masks должны включать оба registers.

`WOD_PUSH_CONSECUTIVE_2`

Кодирует register N и N+1.

Обязательная sanity check:

```
N <= 30
```

Иначе второй register имел бы номер 32 вне 5-bit APX range.

Allocation WOD

`WOD_ALLOC_SMALL`:

```
allocation = (Size + 1) * 8
range 8..128
```

`WOD_ALLOC_LARGE`:

```
allocation = uint16_size * 8
```

`WOD_ALLOC_HUGE`:

```
allocation = raw uint32 byte count
```

При суммировании fixed stack bytes нужно проверять integer overflow. Даже если individual field допустим, сумма нескольких operations может выйти за `uint32_t`.

Save WOD

`WOD_SAVE_NONVOL` использует 16-bit displacement co scale 8.

`WOD_SAVE_NONVOL_FAR` использует 32-bit unscaled displacement.

`WOD_SAVE_XMM128` использует scale 16.

`WOD_SAVE_XMM128_FAR` использует 32-bit unscaled displacement.

Sanity checks должны сопоставлять:

- displacement;

- stack allocation;

- alignment;

- register class;

- operation direction в prolog/epilog.

`WOD_SET_FPREG`

Encoding включает 4-bit register и 4-bit offset.

Смысл:

```
frame_register = RSP + offset * 16
```

В отличие от compact APX integer operations, field register здесь документирован как 4-bit. Parser не должен автоматически расширять его до R16-R31.

`WOD_PUSH_CANONICAL_FRAME`

Operation описывает canonical frame, созданный hardware или OS mechanism. Type различает варианты frame.

До появления стабильных SDK constants parser должен:

- сохранять raw type;

- распознавать документированные значения;

- неизвестное значение отмечать как unsupported/unknown;

- не вычислять stack effect по догадке.

Fragment и chaining

V3 вводит термин **fragment** как непрерывную область machine code, описываемую парой `RUNTIME_FUNCTION`/`UNWIND_INFO_V3`.

Одна source-level функция может иметь:

- main fragment;

- subfragments;

- chained descriptions.

Это усиливает важность различия:

```
source function != ■■■■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■ runtime range
```

Findings overlap и duplicate begin должны учитывать только нормативно допустимую fragment model, а не предполагать простое соответствие один к одному.

Почему текущий проект правильно отказывает V3

Текущий код:

```
version != 1 && version != 2
-> UA_VERSION_UNKNOWN
-> parse failure
```

Это предотвращает:

- чтение `PayloadWords` как `CountOfCodes`;
- чтение `NumberOfOps/NumberOfEpilogs` как frame fields;
- неправильный tail offset;
- фиктивный handler RVA;
- выход за buffer при ложной slot length;
- ложные register masks.

С точки зрения memory safety и semantic honesty такой отказ корректен.

Почему отказ всё равно является продуктовым ограничением

После появления корректных V3 binaries текущая версия:

- не сможет декодировать их metadata;
- создаст `unwind_parse_fail`;
- может увеличить score;
- может потребовать manual review для benign file;
- ухудшит specificity на corpus с V3.

Следовательно, V3 support является compatibility requirement, а не только дополнительной функцией.

Как report должен различать malformed и unsupported

Текущий aggregate finding `unwind_parse_fail` объединяет причины.

Для V3 полезно ввести явное разделение:

```
unsupported_version
malformed_supported_version
```

На уровне per-record report:

```
{
"version": 3,
"parse_status": "unsupported",
"parse_error": "UNWIND_INFO v3 is not supported by this build"
}
```

Unsupported корректный record не должен автоматически получать тот же severity, что truncated V1.

План безопасной реализации V3

Этап 1. Version dispatch

```
switch (version) {
case 1:
case 2:
parse_v1_v2(...);
break;
case 3:
parse_v3(...);
break;
default:
unsupported(...);
}
```

Общие tail/handler helpers можно переиспользовать только после получения правильного version-specific payload end.

Этап 2. Новая модель данных

Нужны:

- 32-bit integer register mask или extensible set;

- epilog summaries;

- count WOD;

- payload size;

- support status;

- version-specific anomaly flags.

Не следует перегружать поля `count_codes`, `frame_reg`, `frame_off` несовместимой семантикой.

Этап 3. Bounded cursor

V3 parser удобнее строить через cursor:

```
payload_begin
payload_end
current
```

Перед каждым чтением:

```
needed <= payload_end - current
```

WOD pool boundaries должны быть вычислены после полного прохода descriptors, но до decoding `FirstOp`.

Этап 4. Descriptor validation

Проверить:

- reserved flags;

- large extension;

- `NumberOfOps`;

- `NumberOfEpilogs`;

- inheritance;

- sign consistency offsets;

- range epilogs внутри fragment;

- `FirstOp` внутри WOD pool;

- WOD count и byte lengths;

- IP offset ordering;

- last instruction boundary.

Этап 5. WOD decoding

Использовать documented dispatch order и отдельную функцию:

```
bool v3_decode_wod(
const uint8_t* pool,
size_t pool_size,
size_t offset,
V3Wod* out,
size_t* consumed);
```

Function не должна изменять cursor при failure.

Этап 6. Summary

Вычислять:

- prolog stack effect;

- saved integer registers;

- saved XMM registers;

- frame register;

- epilog operation references;

- handler/chain.

При переполнении summary counter сохранять raw operation, но ставить anomaly.

Этап 7. Reports и schema

Добавить version-aware fields без удаления V1/V2:

```
encoding_family: "slots_v1_v2" | "wod_v3"
payload_words
prolog_operation_count
epilog_count
```

Schema version должна измениться при изменении публичного contract.

Этап 8. Tests

Минимальный набор:

1. V3 empty payload. 2. One `WOD_ALLOC_SMALL`. 3. APX `WOD_PUSH`. 4. `WOD_PUSH2`. 5. Large prolog. 6. One full epilog. 7. Inherited epilog. 8. Large epilog. 9. Shared WOD region. 10. Handler tail. 11. Chained fragment. 12. Invalid reserved flag. 13. Truncated WOD. 14. `FirstOp` OOB. 15. Invalid inheritance. 16. Mixed epilog offset signs. 17. Register 31 consecutive-pair error. 18. Differential sample from `ml64 /unwindv3`.

Этап 9. Corpus gate

До объявления поддержки нужны:

- compiler-generated V3 objects;
- linked PE images;
- comparison с Microsoft/LLVM tools, когда они поддерживают вывод;
- benign zero-false-positive corpus;
- mutation/fuzz seeds V3;
- backward regression V1/V2.

Как V3 влияет на 27 findings

Без изменений остаются PE layout и function table findings.

Требуют version-aware logic:

- `unwind_parse_fail`;
- `unwind_anomalies`;
- placement findings, если parser должен получить handler;
- handler dominance;
- combo findings.

Reuse по `UnwindInfoRVA` можно считать до полного decode, но интерпретация должна учитывать fragment sharing.

Технически корректный ответ о V3

> Текущая версия безопасно поддерживает `UNWIND_INFO` V1/V2 и намеренно > отказывает неизвестную V3, чтобы не интерпретировать её по старому layout. > Microsoft опубликовала V3 как preview specification для Intel APX в мае > 2026 года. V3 сохраняет `.pdata` и 12-byte `RUNTIME_FUNCTION`, но вводит > `PayloadWords`, явные epilog descriptors и variable-length WOD pool. > Поэтому поддержка требует отдельного decoder, расширенной модели registers, > новых tests и обновления report schema. До этого корректные V3 files > являются известной границей совместимости и могут создавать parse-failure > signals.

Нормативный источник

Microsoft Learn, `Exception Handling Unwind Information V3`:

`https://learn.microsoft.com/en-us/cpp/build/x64-unwind-information-v3?view=msvc-170`

Microsoft Learn, conventional x64 exception handling V1:

`https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170`

Контрольные вопросы

1. Почему одинаковый размер header не делает V3 совместимой с V1/V2? 2. Что содержит `PayloadWords` и чего оно не содержит? 3. Почему WOD decoder должен проверять opcodes в определённом порядке? 4. Для чего нужны explicit epilog descriptors? 5. Почему текущего 16-bit register mask недостаточно? 6. Чем large prolog отличается от large epilog? 7. Почему unsupported V3 нельзя считать malformed автоматически? 8. Какие существующие findings остаются применимыми без изменений? 9. Какие поля JSON должны стать version-aware? 10. Какой test corpus нужен перед объявлением поддержки V3?

Приложение A. Связный терминологический указатель

Как пользоваться указателем

Определения ниже не заменяют основные лекции. Они предназначены для повторения. Термин следует понимать вместе с указанным отношением к соседним понятиям и проекту.

Программа, файл и исполнение

Source code - текст программы на языке программирования. Для C он проходит preprocessing и compilation. В проекте source code находится в `src/`, `include/` и tests.

Preprocessor - стадия C translation, обрабатывающая `#include`, macros и conditional compilation. Благодаря `#ifdef _WIN32` один source file содержит Windows implementation и non-Windows stub.

Compiler - программа, переводящая translation units в object code и выполняющая diagnostics/optimization. Clang, GCC и MSVC поддерживаются build system в разных средах.

Object file - результат отдельной compilation unit до окончательного linking. Содержит machine code, data, symbols и relocations, но ещё не является готовым executable image.

Linker - компоновщик, объединяющий object files и libraries, разрешающий symbols и создающий PE image.

Executable image - файл, предназначенный для отображения loader в memory; в Windows обычно EXE или DLL в формате PE.

Loader - часть runtime system, отображающая image sections, применяющая relocations, разрешающая imports и подготавливающая process execution.

Process - выполняющийся экземпляр программы с virtual address space, handles, modules и threads.

Thread - единица исполнения внутри process, имеющая собственные registers, stack и instruction pointer.

Module - загруженный PE image внутри process: main EXE или DLL.

Host - операционная система и machine, на которых работает analyzer.

Target или **debuggee** - программа, которую analyzer исследует или запускает под debugger.

Данные и адреса

Bit - минимальная binary unit со значением 0 или 1.

Byte - в рассматриваемых платформах восемь bits.

Unsigned integer - целое без знака; все bit combinations выражают nonnegative value.

Signed integer - целое со знаком; на современных systems обычно two's complement.

Overflow - результат arithmetic вне диапазона типа. Для unsigned C он вычисляется modulo power of two; для signed overflow поведение undefined.

Endianness - порядок bytes многобайтового значения. PE AMD64 использует little-endian.

Little-endian - least significant byte хранится по меньшему адресу.

Pointer - typed C value, обозначающее address object/function в адресном пространстве текущего процесса.

File offset - позиция byte в файле на диске.

Virtual Address, VA - адрес в virtual address space process.

Relative Virtual Address, RVA - VA относительно module base: `RVA = VA - base`.

ImageBase - preferred base PE image из optional header.

Actual module base - фактический base загруженного module; может отличаться из-за ASLR.

ASLR - Address Space Layout Randomization, выбор изменяемых bases для усложнения предсказания addresses.

Address range - interval addresses. В проекте функции представлены полуинтервалом `[begin,end)`.

Out of bounds, OOB - address или range выходит за declared boundary, например file size или SizeOfImage.

Alignment - требование кратности address/offset определённому числу bytes.

Saturation - ограничение результата maximum/minimum вместо wraparound; используется при aggregate stack size.

CPU и ABI

Instruction Set Architecture, ISA - модель instructions, registers, address sizes и архитектурных effects. Проект ориентирован на AMD64/x86-64 target.

Machine code - bytes instructions конкретной ISA.

Assembly language - textual representation machine instructions.

Register - быстрое архитектурное состояние CPU.

Instruction pointer - register address следующей/текущей instruction; на x86-64 `RIP`.

Stack pointer - register вершины stack; на x86-64 `RSP`.

Flags register - register condition/control bits; в context используется `EFlags`.

Application Binary Interface, ABI - правила двоичного взаимодействия: argument registers, preserved registers, stack alignment, object formats и unwind conventions.

Calling convention - часть ABI, описывающая передачу arguments, return value и register preservation.

Volatile register - register, который callee может изменить без восстановления.

Nonvolatile register - register, который callee обязан сохранить и восстановить, если использует.

Shadow space - 32-byte область, резервируемая caller в Windows x64 для первых четырёх register arguments.

Stack alignment - требование к кратности `RSP` в определённых execution points.

Function - region code с agreed entry/return semantics.

Caller - function, выполняющая call.

Callee - вызываемая function.

Prolog - ограниченная начальная sequence, создающая stack frame и сохраняющая nonvolatile state.

Epilog - sequence, отменяющая frame effects и возвращающая control caller.

Stack frame - часть thread stack и register relation, принадлежащая одному active function invocation.

Frame pointer/register - nonvolatile register, используемый как стабильная reference point frame.

Leaf function - функция, для которой Windows может выполнить простую unwind без table entry при соблюдении ABI constraints.

Nonleaf function - функция, обычно требующая unwind metadata из-за calls, stack allocation или nonvolatile state.

Context - snapshot выбранного архитектурного состояния thread.

PE/COFF

PE - Portable Executable, Windows image format.

COFF - Common Object File Format, структура object/image headers и related records.

DOS header - начальный compatibility header PE с signature `MZ`.

`e_lfanew` - field по offset `0x3C`, указывающее file offset PE signature.

PE signature - bytes `PE\0\0`.

COFF file header - header с Machine, NumberOfSections и SizeOfOptionalHeader.

Optional header - обязательный для image header с ImageBase, sizes, alignments и data directories.

PE32+ - 64-bit variant optional header, magic `0x20B`.

Machine type - architecture identifier; AMD64 имеет value `0x8664`.

Section - basic unit code/data mapping в PE.

Section header - record с name, virtual/raw ranges и characteristics.

VirtualSize - значимый virtual size section.

SizeOfRawData - размер file-backed section bytes.

PointerToRawData - file offset начала raw section data.

Characteristics - bit flags назначения/permissions section.

Data directory - RVA+size системной table в optional header.

Exception directory - data directory, указывающая AMD64 runtime function table.

`.text` - распространённое имя executable code section; имя не является жёстким нормативным условием.

`.pdata` - распространённое имя section runtime function entries.

`.xdata` - распространённое имя section unwind records.

`.bss` - распространённое имя zero-initialized virtual data без обязательных raw bytes.

`.idata` - распространённое имя import data.

SizeOfImage - declared size загруженного image в RVA space.

SizeOfHeaders - file/memory extent PE headers.

SectionAlignment - alignment virtual sections.

FileAlignment - alignment raw section data.

Исключения и unwind

Exception - событие, прерывающее normal instruction flow и инициирующее Windows dispatch.

Exception code - numeric identifier класса exception.

Structured Exception Handling, SEH - Windows mechanism поиска и обработки exceptions.

Dispatch - поиск подходящего handler.

Unwind - восстановление caller states и сворачивание frames.

Virtual unwind - вычисление caller context без исполнения original epilog.

Search phase - проход frames для выбора handler.

Unwind phase - фактическое сворачивание до выбранной точки.

`RUNTIME_FUNCTION` - AMD64 record `{BeginRVA, EndRVA, UnwindRVA}`.

Function table - sorted array `RUNTIME_FUNCTION`.

`UNWIND_INFO` - variable-length record effects prolog, handler или chain.

Version - format version в low three bits header.

Flags - high five bits header, включая EHANDLER, UHANDLER и CHAININFO.

`SizeOfProlog` - длина declared prolog в bytes.

`CountOfCodes` - число two-byte unwind slots, не число logical operations.

`UNWIND_CODE` - two-byte slot с CodeOffset и opcode/OpInfo.

CodeOffset - offset в prolog, связанный с operation.

Opcode - тип unwind operation.

Operand - дополнительное encoded value operation.

`UWOP_PUSH_NONVOL` - описание push nonvolatile register.

`UWOP_ALLOC_SMALL` - compact small stack allocation.

`UWOP_ALLOC_LARGE` - stack allocation с additional operands.

`UWOP_SET_FPREG` - установка frame register.

`UWOP_SAVE_NONVOL` - сохранение GP nonvolatile register в stack slot.

`UWOP_SAVE_XMM128` - сохранение nonvolatile XMM register.

`UWOP_PUSH_MACHFRAME` - описание system machine frame.

Handler RVA - image-relative адрес frame handler.

Chained unwind info - record, ссылающаяся на primary runtime function/unwind description.

Frame-based handler - handler, связанный с stack frame через unwind metadata.

VEH - Vectored Exception Handler, process-level registration до frame search.

VCH - Vectored Continue Handler, vectored continuation mechanism.

UEF - Unhandled Exception Filter, top-level process filter.

Dynamic function table - runtime-registered table для generated code.

JIT compiler - Just-In-Time compiler, создающий machine code во время process execution.

Fragment в Unwind V3 - непрерывный диапазон machine code, описанный одной парой `RUNTIME_FUNCTION` и `UNWIND_INFO_V3`.

WOD - Winding Operation Descriptor, variable-length descriptor одной операции unwind в V3.

WOD pool - packed byte array всех WOD prolog и epilog одного V3 fragment.

PayloadWords - число 16-bit words payload V3 после четырёхбайтового header; handler или chained record в это число не входят.

Debugging

Debugger - process/tool, управляющий target через Debug API.

Debug event - Windows notification о create/exit, DLL load или exception.

`WaitForDebugEvent` - ожидание следующего event.

`ContinueDebugEvent` - продолжение target с указанным status.

First chance - первое уведомление debugger до окончательной application handling.

Second chance - уведомление после неуспешного normal handling.

`DBG_CONTINUE` - debugger объявляет event обработанным.

`DBG_EXCEPTION_NOT_HANDLED` - debugger передаёт ordinary exception дальнейшей системе dispatch.

Breakpoint - контролируемая остановка execution по адресу/условию.

Software breakpoint - замена instruction byte на trap instruction.

`INT3` - однобайтовая x86 breakpoint instruction `0xCC`.

Original byte - byte target code, сохранённый до вставки `INT3`.

Re-arm - повторная установка breakpoint после исполнения original instruction.

Trap Flag, TF - bit `EFlags`, вызывающий single-step exception после одной instruction.

Single-step - режим пошагового исполнения.

Pending re-arm - association thread с breakpoint, который нужно восстановить на следующем single-step.

Remote process memory - память target, читаемая/изменяемая analyzer через Debug API.

Instruction cache - CPU cache fetched instructions; после code modification требуется flush.

Timeout - верхняя граница времени dynamic session, но не security sandbox.

Trace - в проекте bounded sequence RIP после handler hit, не полный execution log.

Анализ и доказательства

Parser - component, преобразующий bytes в structured fields с bounds checks.

Decoder - component, интерпретирующий encoded domain structure, например unwind operations.

Invariant - условие, которое должно сохраняться для корректной model.

Observation - непосредственно полученный факт конкретного file/run.

Finding - named analytical record, созданный rule из observations.

Severity - qualitative importance finding: medium/high и т. п.

Evidence family - группа related findings одного analytical слоя.

Score - bounded engineering sum weights 0-100.

Suspicion - в проекте `score / 100`, не probability.

Confidence - rule-based оценка силы/разнообразия evidence, не эмпирически калиброванная probability.

Verdict - categorical label по thresholds.

Weakening rule - правило, снижающее confidence при benign alternatives или недостатке context.

Mitigation - internal C name weakening rule; не исправляет input.

Correlation - relation static и dynamic evidence. Aggregate correlation слабее exact address-level identity.

False positive - benign object, ошибочно classified suspicious.

False negative - suspicious behavior, не обнаруженное analyzer.

Ground truth - эталонная разметка, относительно которой проверяется правильность результата.

True positive, TP - размеченное нарушение, которое инструмент обнаружил.

True negative, TN - корректный объект, который инструмент не отметил положительным сигналом.

Sensitivity / Recall - доля обнаруженных положительных примеров: `TP / (TP + FN)`.

Specificity - доля правильно отклонённых отрицательных примеров: `TN / (TN + FP)`.

Precision - доля подтверждённых сигналов среди всех положительных сигналов: `TP / (TP + FP)`.

F1 - гармоническое среднее precision и recall.

Confidence interval - диапазон статистически правдоподобных значений параметра при заданной модели выборки; не устраняет смещение corpus.

Calibration - подбор/проверка thresholds на размеченном corpus.

Corpus - набор samples для testing/measurement.

Regression test - test, сохраняющий ранее установленное expected behavior.

Unit test - проверка отдельного module/contract.

Integration test - проверка взаимодействия components и external outputs.

Fuzzing - генерация множества inputs для поиска crashes и unsafe paths.

Mutation fuzzing - изменение seed inputs.

Coverage-guided fuzzing - генерация inputs с использованием code coverage feedback.

ASan - AddressSanitizer.

UBSan - UndefinedBehaviorSanitizer.

Reproducibility - возможность повторить analysis с тем же input, versions и commands.

Отчёты и интеграция

Serialization - преобразование in-memory model во внешний format.

CSV - flat tabular text format.

JSON - structured text format objects/arrays/scalars.

HTML - document markup; report является self-contained dashboard.

Schema - formal description allowed JSON structure and types.

Schema version - version public data contract.

Provenance - сведения о происхождении result: input hash, tool/build, time, command.

SHA-256 - 256-bit cryptographic hash, используемый для identity bytes.

Atomic write - запись temporary file и rename, предотвращающие наблюдение partial result.

UTF-8 - variable-length Unicode encoding, валидируемая report writer.

Escaping - преобразование special characters для безопасного JSON/HTML representation.

Content Security Policy, CSP - browser policy, ограничивающая sources content в HTML report.

Backend - server-side application, управляющее upload, jobs, core execution и persistence.

Frontend - user interface, взаимодействующее с backend API.

API - формальный интерфейс между components. CLI+JSON также является API, хотя не network API.

Subprocess - отдельный OS process, запускаемый backend для analysis.

Job queue - механизм асинхронного планирования analyses.

Database - persistent structured storage jobs/reports.

Raw JSON - неизменённый full report; рекомендуется сохранять как source of truth.

Normalization - выделение indexed fields JSON в relational/search structures.

Sandbox - изолированная execution environment. Debugger и timeout сами по себе sandbox не образуют.

Release candidate, RC - предварительный release, близкий к stable, но ещё допускающий blocking changes.

Приложение B. Нормативные и технические источники

Microsoft

1. [PE Format](https://learn.microsoft.com/en-us/windows/win32/debug/pe-format) - PE/COFF headers, sections, RVA, directories и `.pdata`. 2. [x64 exception handling](https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170) - `RUNTIME_FUNCTION`, `UNWIND_INFO`, operations и unwind procedure. 3. [x64 Unwind Information V3](https://learn.microsoft.com/en-us/cpp/build/x64-unwind-information-v3?view=msvc-170) - preview layout V3, explicit epilogs, WOD pool и Intel APX registers. 4. [x64 calling convention](https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention?view=msvc-170) - registers, arguments, alignment и unwindability. 5. [x64 prolog and epilog](https://learn.microsoft.com/en-us/cpp/build/prolog-and-epilog?view=msvc-170) - ограничения prolog/epilog. 6. [Structured Exception Handling](https://learn.microsoft.com/en-us/windows/win32/debug/structured-exception-handling) - общая модель SEH. 7. [Debugging Events](https://learn.microsoft.com/en-us/windows/win32/debug/debugging-events) - lifecycle debug events. 8. [Writing the Debugger's Main Loop](https://learn.microsoft.com/en-us/windows/win32/debug/writing-the-debugger-s-main-loop) - `WaitForDebugEvent`/`ContinueDebugEvent`. 9. [AddVectoredExceptionHandler](https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-addvectoredexceptionhandler). 10. [SetUnhandledExceptionFilter](https://learn.microsoft.com/en-us/windows/win32/api/errhandlingapi/nf-errhandlingapi-setunhandledexceptionfilter). 11. [RtlAddFunctionTable](https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable). 12. [RtlAddGrowableFunctionTable](https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdgrowablefunctiontable). 13. [WaitForDebugEventEx](https://learn.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-waitfordebugeventex).

Стандарты и документация tools

1. ISO/IEC 9899:2018 - язык C17. Текст стандарта может распространяться на условиях ISO; для практического reference используется документация compiler и committee drafts. 2. AMD64 Architecture Programmer's Manual - architecture registers, instructions и exception model. 3. Intel 64 and IA-32 Architectures Software Developer's Manual - x86-64 instruction and system semantics. 4. [CMake Documentation](https://cmake.org/cmake/help/latest/) - targets, presets, CTest и CPack. 5. [Ghidra Documentation](https://ghidra-sre.org/) - import, disassembly, decompiler и scripting. 6. [JSON Schema](https://json-schema.org/specification) - формальная валидация JSON documents. 7. [NIST Static Analysis Tool Study Methodology](https://www.nist.gov/system/files/documents/2021/03/24/CAS%202012%20Static%20Analysis%20Tool%20Study%20Methodology.pdf) - различие accuracy, precision и recall при оценке analysis tools. 8. [NIST/SEMATECH e-Handbook of Statistical Methods](https://www.itl.nist.gov/div898/handbook/) - основы статистической оценки и confidence intervals.

Приложение C. Карта ответов на защите

Что делает проект

Проект анализирует Windows PE32+ AMD64 files. Он безопасно читает PE layout, извлекает exception directory, декодирует `RUNTIME_FUNCTION` и `UNWIND_INFO`, выявляет structural/placement/reuse anomalies, а на Windows может запустить target под Debug API и наблюдать exception-related registrations, handler hits и dynamic function tables. Результаты выводятся в terminal, CSV, JSON и HTML.

Почему это нужно

EH/unwind metadata требуется Windows для exception dispatch, stack unwinding и debugging. Она также содержит function boundaries, stack effects и handler relations, полезные reverse engineer даже без symbols. Повреждённая или необычно организованная metadata может указывать на corruption, custom runtime, JIT, obfuscation или exception-driven control flow.

В чём научно-техническая задача

Задача состоит не в чтении одного поля PE, а в построении безопасного and explainable pipeline:

```
untrusted bytes
-> validated structural model
-> decoded ABI metadata
-> evidence aggregation
-> optional runtime observations
-> reproducible reports
```

Почему архитектура не microservices

Core - modular monolith: static C library плюс CLI. Для одного analytical tool это уменьшает deployment complexity и сохраняет чёткие module boundaries. Backend, database, queue и Windows dynamic workers могут быть отдельными services вокруг stable JSON/CLI contract.

Почему static работает на macOS/Linux

Static parser не исполняет input и не вызывает Windows loader. Он интерпретирует bytes по documented PE format. Dynamic mode требует actual Windows Debug API и поэтому Windows-only.

Почему score не probability

Weights и thresholds заданы engineering rules. Нет обученной model и empirical probability calibration. Score ранжирует anomaly evidence; confidence оценивает разнообразие и strength rules.

Что означает successful analysis

Input прочитан; минимальная PE32+ AMD64 model извлечена; requested stages завершились; outputs записаны. Это не означает, что file benign или что все возможные behaviors исследованы.

Что является главным ограничением

Static stage не сравнивает полностью metadata с machine-code prolog и не анализирует весь binary semantics. Dynamic stage покрывает только достигнутый path и должен запускаться в isolated Windows environment.

Можно ли подключить frontend и database без изменения core

Да. Backend запускает CLI как subprocess, валидирует JSON schema и сохраняет raw report. Core changes нужны только для новых analytical fields или stable native API, но не для обычной web integration.

Почему проект готов к ВКР

Есть работающий core, объяснимая архитектура, documented standards, static and dynamic modes, structured reports, Ghidra integration, strict build, 15 automated CTest cases, fuzzing, synthetic and real-corpus validation и явно сформулированные limitations.

Что развивать после ВКР

Приоритеты:

1. Windows sandbox worker. 2. Remote decoding dynamic function tables. 3. Code-to-unwind correspondence. 4. Broader labeled corpus and calibration. 5. Backend/database/frontend. 6. Versioned heuristic policy. 7. Richer typed event schema.

Часть VI. Большой практикум по чтению признаков

Этот практикум нужен, чтобы знания не остались набором терминов. Здесь один и тот же аналитический путь повторяется в разных ситуациях: открыть отчет, найти finding, понять affected count, перейти к representative RVA, проверить Ghidra и сформулировать вывод.

Практикум 1. преподаватель спрашивает, почему affected count больше finding count

Представь, что в отчете фигурирует pe_section_raw_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 2. в HTML в Runtime functions повторяется одно и то же предупреждение

Представь, что в отчете фигурирует pe_section_virtual_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: виртуальный диапазон секции выходит за объявленный SizeOfImage. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки

разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 3. JSON показывает нулевые counters, а HTML их не показывает

Представь, что в отчете фигурирует pe_section_raw_overlap. Не начинай ответ с verdict. Начинай с наблюдаемого факта: две секции пересекаются в файле и могут трактовать одни байты по-разному. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 4. Status OK стоит рядом с placement warning

Представь, что в отчете фигурирует pe_section_virtual_overlap. Не начинай ответ с verdict. Начинай с наблюдаемого факта: две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки

разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 5. dynamic mode показал RtlAddFunctionTable, но handler hit равен нулю

Представь, что в отчете фигурирует pe_section_misaligned. Не начинай ответ с verdict. Начинай с наблюдаемого факта: адреса или размеры секций не согласованы с file/section alignment. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 6. Ghidra уже создала функции до импорта CSV

Представь, что в отчете фигурирует pe_exception_directory_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: Exception Directory находится вне допустимого образа или не переводится в валидные байты. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки

разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 7. адрес в отчете выглядит как RVA, а Ghidra показывает другое число

Представь, что в отчете фигурирует no_runtime_functions. Не начинай ответ с verdict. Начинай с наблюдаемого факта: для AMD64 образа не найдено записей RUNTIME_FUNCTION. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 8. нужно объяснить, почему score не является вероятностью

Представь, что в отчете фигурирует invalid_ranges. Не начинай ответ с verdict. Начинай с наблюдаемого факта: в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 9. нужно доказать, что проект не является антивирусом

Представь, что в отчете фигурирует rf_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: диапазон функции выходит за границы валидного image/section context. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 10. нужно быстро выбрать лучший пример для презентации

Представь, что в отчете фигурирует unwind_rva_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: адрес UNWIND_INFO не отображается в валидный file offset или секцию. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 11. преподаватель спрашивает, почему affected count больше finding count

Представь, что в отчете фигурирует rf_unsorted. Не начинай ответ с verdict. Начинай с наблюдаемого факта: таблица функций не отсортирована по BeginAddress. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 12. в HTML в Runtime functions повторяется одно и то же предупреждение

Представь, что в отчете фигурирует rf_overlaps. Не начинай ответ с verdict. Начинай с наблюдаемого факта: диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 13. JSON показывает нулевые counters, а HTML их не показывает

Представь, что в отчете фигурирует duplicate_begin. Не начинай ответ с verdict. Начинай с наблюдаемого факта: несколько записей имеют одинаковое начало функции. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 14. Status OK стоит рядом с placement warning

Представь, что в отчете фигурирует unwind_parse_fail. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO невозможно разобрать до конца по формату Windows x64. Затем назови family unwind_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 15. dynamic mode показал RtlAddFunctionTable, но handler hit равен нулю

Представь, что в отчете фигурирует unwind_anomalies. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern. Затем назови family unwind_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 16. Ghidra уже создала функции до импорта CSV

Представь, что в отчете фигурирует unwind_reuse_moderate. Не начинай ответ с verdict. Начинай с наблюдаемого факта: заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов. Затем назови family unwind_reuse. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 17. адрес в отчете выглядит как RVA, а Ghidra показывает другое число

Представь, что в отчете фигурирует unwind_reuse_strong. Не начинай ответ с verdict. Начинай с наблюдаемого факта: слишком большая доля функций использует один и тот же UnwindInfoRVA. Затем назови family unwind_reuse. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 18. нужно объяснить, почему score не является вероятностью

Представь, что в отчете фигурирует handler_dominance. Не начинай ответ с verdict. Начинай с наблюдаемого факта: один handler target обслуживает подозрительно много функций. Затем назови family handler_dominance. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на обработчики исключений. Если один handler обслуживает слишком много функций, аналитик должен понять, это нормальный runtime helper или централизованный exception dispatcher.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 19. нужно доказать, что проект не является антивирусом

Представь, что в отчете фигурирует unwind_in_exec_section. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код, а не metadata. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 20. нужно быстро выбрать лучший пример для презентации

Представь, что в отчете фигурирует unwind_unusual_section. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO лежит не в типичной секции .xdata/.rdata/.pdata или близком data-аналоге. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 21. преподаватель спрашивает, почему affected count больше finding count

Представь, что в отчете фигурирует unwind_writable_section. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO лежит в секции с правом записи. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 22. в HTML в Runtime functions повторяется одно и то же предупреждение

Представь, что в отчете фигурирует handler_nonexec_section. Не начинай ответ с verdict. Начинай с наблюдаемого факта: handler RVA указывает в неисполняемую или неизвестную секцию. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 23. JSON показывает нулевые counters, а HTML их не показывает

Представь, что в отчете фигурирует handler_writable_section. Не начинай ответ с verdict. Начинай с наблюдаемого факта: handler RVA указывает в writable секцию. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 24. Status OK стоит рядом с placement warning

Представь, что в отчете фигурирует placement_exec_nonexec_handler_combo. Не начинай ответ с verdict. Начинай с наблюдаемого факта: одновременно странно размещены unwind metadata и handler target. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 25. dynamic mode показал RtlAddFunctionTable, но handler hit равен нулю

Представь, что в отчете фигурирует placement_integrity_combo. Не начинай ответ с verdict. Начинай с наблюдаемого факта: placement anomaly подтверждается повреждением или странностью UNWIND_INFO. Затем назови family unwind_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 26. Ghidra уже создала функции до импорта CSV

Представь, что в отчете фигурирует reuse_dominance_combo. Не начинай ответ с verdict. Начинай с наблюдаемого факта: массовое повторное использование unwind metadata сочетается с доминированием handler. Затем назови family unwind_reuse. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 27. адрес в отчете выглядит как RVA, а Ghidra показывает другое число

Представь, что в отчете фигурирует writable_metadata_handler_combo. Не начинай ответ с verdict. Начинай с наблюдаемого факта: и unwind metadata, и handler target связаны с writable context. Затем назови family metadata_placement. Это сразу показывает, к какому слою относится проблема: Эта семья признаков спрашивает: где физически лежат unwind metadata и handler targets. Для нормального compiler output metadata обычно находится в read-only data секциях, а handler - в executable code.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 28. нужно объяснить, почему score не является вероятностью

Представь, что в отчете фигурирует pe_section_raw_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: секция ссылается на байты за пределами файла, поэтому чтение raw data становится недостоверным. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 29. нужно доказать, что проект не является антивирусом

Представь, что в отчете фигурирует pe_section_virtual_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: виртуальный диапазон секции выходит за объявленный SizeOfImage. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 30. нужно быстро выбрать лучший пример для презентации

Представь, что в отчете фигурирует pe_section_raw_overlap. Не начинай ответ с verdict. Начинай с наблюдаемого факта: две секции пересекаются в файле и могут трактовать одни байты по-разному. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 31. преподаватель спрашивает, почему affected count больше finding count

Представь, что в отчете фигурирует pe_section_virtual_overlap. Не начинай ответ с verdict. Начинай с наблюдаемого факта: две секции пересекаются в виртуальной памяти, создавая неоднозначность RVA. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 32. в HTML в Runtime functions повторяется одно и то же предупреждение

Представь, что в отчете фигурирует pe_section_misaligned. Не начинай ответ с verdict. Начинай с наблюдаемого факта: адреса или размеры секций не согласованы с file/section alignment. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 33. JSON показывает нулевые counters, а HTML их не показывает

Представь, что в отчете фигурирует pe_exception_directory_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: Exception Directory находится вне допустимого образа или не переводится в валидные байты. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 34. Status OK стоит рядом с placement warning

Представь, что в отчете фигурирует no_runtime_functions. Не начинай ответ с verdict. Начинай с наблюдаемого факта: для AMD64 образа не найдено записей RUNTIME_FUNCTION. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 35. dynamic mode показал RtlAddFunctionTable, но handler hit равен нулю

Представь, что в отчете фигурирует invalid_ranges. Не начинай ответ с verdict. Начинай с наблюдаемого факта: в RUNTIME_FUNCTION начало, конец и размер функции образуют невозможный диапазон. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 36. Ghidra уже создала функции до импорта CSV

Представь, что в отчете фигурирует rf_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: диапазон функции выходит за границы валидного image/section context. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 37. адрес в отчете выглядит как RVA, а Ghidra показывает другое число

Представь, что в отчете фигурирует unwind_rva_oob. Не начинай ответ с verdict. Начинай с наблюдаемого факта: адрес UNWIND_INFO не отображается в валидный file offset или секцию. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 38. нужно объяснить, почему score не является вероятностью

Представь, что в отчете фигурирует rf_unsorted. Не начинай ответ с verdict. Начинай с наблюдаемого факта: таблица функций не отсортирована по BeginAddress. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 39. нужно доказать, что проект не является антивирусом

Представь, что в отчете фигурирует rf_overlaps. Не начинай ответ с verdict. Начинай с наблюдаемого факта: диапазоны функций пересекаются и создают неоднозначное соответствие адреса функции. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 40. нужно быстро выбрать лучший пример для презентации

Представь, что в отчете фигурирует duplicate_begin. Не начинай ответ с verdict. Начинай с наблюдаемого факта: несколько записей имеют одинаковое начало функции. Затем назови family table_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет фундамент: правильно ли PE-файл описывает свои секции и таблицу функций. Если на этом уровне есть ошибка, более высокие выводы надо читать осторожно, потому что адреса могут быть недостоверны.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 41. преподаватель спрашивает, почему affected count больше finding count

Представь, что в отчете фигурирует unwind_parse_fail. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO невозможно разобрать до конца по формату Windows x64. Затем назови family unwind_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

```
$json = Get-Content ".\reports\combo_28_dynamic.json" -Raw | ConvertFrom-Json
$json.heuristic_analysis.findings | Format-Table
code,family,severity,affected_count,representative_rva -AutoSize
$json.dynamic_validation.events | Format-Table type,module,address,arg0,arg1,arg2,arg3
-AutoSize
```

Практикум 42. в HTML в Runtime functions повторяется одно и то же предупреждение

Представь, что в отчете фигурирует unwind_anomalies. Не начинай ответ с verdict. Начинай с наблюдаемого факта: UNWIND_INFO читается, но содержит странности в version, opcode, operand, chain или ABI pattern. Затем назови family unwind_integrity. Это сразу показывает, к какому слою относится проблема: Эта семья признаков проверяет внутреннее устройство UNWIND_INFO. Здесь важны длины, поля, коды операций, хвост структуры и согласованность с форматом Windows x64.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 43. JSON показывает нулевые counters, а HTML их не показывает

Представь, что в отчете фигурирует unwind_reuse_moderate. Не начинай ответ с verdict. Начинай с наблюдаемого факта: заметная доля функций использует один и тот же UnwindInfoRVA при наличии других сигналов. Затем назови family unwind_reuse. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 44. Status OK стоит рядом с placement warning

Представь, что в отчете фигурирует unwind_reuse_strong. Не начинай ответ с verdict. Начинай с наблюдаемого факта: слишком большая доля функций использует один и тот же UnwindInfoRVA. Затем назови family unwind_reuse. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на статистику повторного использования одного UnwindInfoRVA. Повтор сам по себе не всегда плох, но массовый повтор может скрывать ручную таблицу или низкоинформативную metadata.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.

Практикум 45. dynamic mode показал RtlAddFunctionTable, но handler hit равен нулю

Представь, что в отчете фигурирует handler_dominance. Не начинай ответ с verdict. Начинай с наблюдаемого факта: один handler target обслуживает подозрительно много функций. Затем назови family handler_dominance. Это сразу показывает, к какому слою относится проблема: Эта семья признаков смотрит на обработчики исключений. Если один handler обслуживает слишком много функций, аналитик должен понять, это нормальный runtime helper или централизованный exception dispatcher.

Следующий шаг - масштаб. Если affected count равен 1, ты показываешь одну запись или один адрес. Если affected count равен 46, как в combo_28, ты объясняешь массовый pattern. Это принципиально: массовость усиливает важность, но не превращает один code в десятки разных codes.

После масштаба переходи к месту. Representative RVA - это стартовая точка ручной проверки. В Ghidra его надо перевести через ImageBase. В PE viewer надо проверить секцию, raw offset, virtual size и permissions. Если finding связан с UNWIND_INFO, открой bytes metadata. Если связан с handler, открой handler target.

Финальный ответ должен звучать спокойно: 'Analyzer нашел объяснимый static finding. Он показывает такой-то низкоуровневый факт. Для окончательной интерпретации я проверяю такой-то адрес и сопоставляю его с dynamic evidence'. Такой ответ сильнее, чем попытка сказать 'программа точно вредоносная'.